COMP 3613 Assignment 1

Individual report

Field	Value
Student name	Gian Suarez Nápoles
Student ID	816042658
Deployed app	https://faststarter-07fs.onrender.com
Logins	bob / bobpass: student (seeded; Submit Application) · admin / adminpass: admin (Match Application, Add company) · comp1 / comppass: company (Publish/Manage Internships). Not seeded: created on the live site with the admin's Add company.
GitHub	https://github.com/Gianmarcos794/Comp3613_A1
YouTube	https://youtu.be/RPX4wAlBhw8
Agent	Claude Code (VS Code extension), with the student-build and student-judge skills; Render MCP for deployment.
Skill integrity	PASS. Root e84cd692d0b8...ccff5c8beb

The YouTube video must show this name and must not show or say this ID. App logins belong in this report. Database passwords do not.

Draft this file with the Guide. **Update it after every phase milestone** before you pause. The use-case diagram is a UML PNG at docs/diagrams/use-case.png, linked from this file as diagrams/use-case.png (path relative to docs/report.md). The model diagram is Mermaid. **Embed wireframe images** as wireframes/<file> (files live in docs/wireframes/).

Do not put your student ID in this file if you will commit it. The PDF cover adds your name and ID at export time.

Assigned project

Internship Platform - a departmental system that takes student applications and distributes / matches them across internship positions offered by participating companies.

Interpretation

InternSearch treats internships as a **matching service run by the department**, not a job board.

• **Students apply to a general pool**, not to a specific posting. Each student has one application: their profile details (program, year, GPA), the kind of role they want, and their projects. The student sees its status: submitted, matched (with the internship title, company and location), or rejected. A rejected application can be edited and resubmitted.
• **Companies are trusted partners.** They cannot register themselves; the Admin creates each company account once the company is approved. A company publishes internships with requirements (program, minimum GPA, minimum year, number of positions, deadline), manages or closes them, and sees the students matched to them.
• **The Admin makes the match, and it is final.** The system always checks requirements first (program, GPA, year, open status, deadline not passed), so the Admin only picks from internships the student is eligible for. If none fit, the Admin rejects the application.
• **Rules that keep it fair:** student sign-up needs a .edu university email, GPA is capped at 4.3, an internship closes when its positions are filled, and an internship with matches can only be closed, never deleted.

Three workflows

Three workflows named by the student (three is the minimum), as Feature (user). Screen steps are shown in the Phase 4 wireframes.

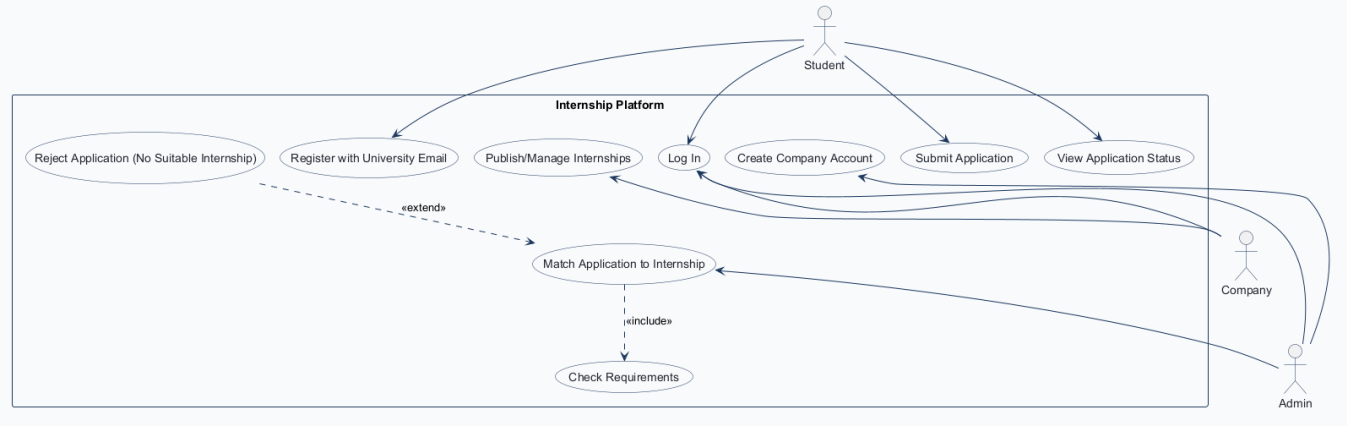
1. Publish/Manage Internships (Company)

2. Submit Application (Student)

3. Match Application to Internship (Admin)

Review Application (Company) was removed after Phase 2/3 because the Admin's match is final.

Use case diagram



Source: docs/diagrams/use-case.json (rendered with python manage.py usecase).

Actors and their use cases:

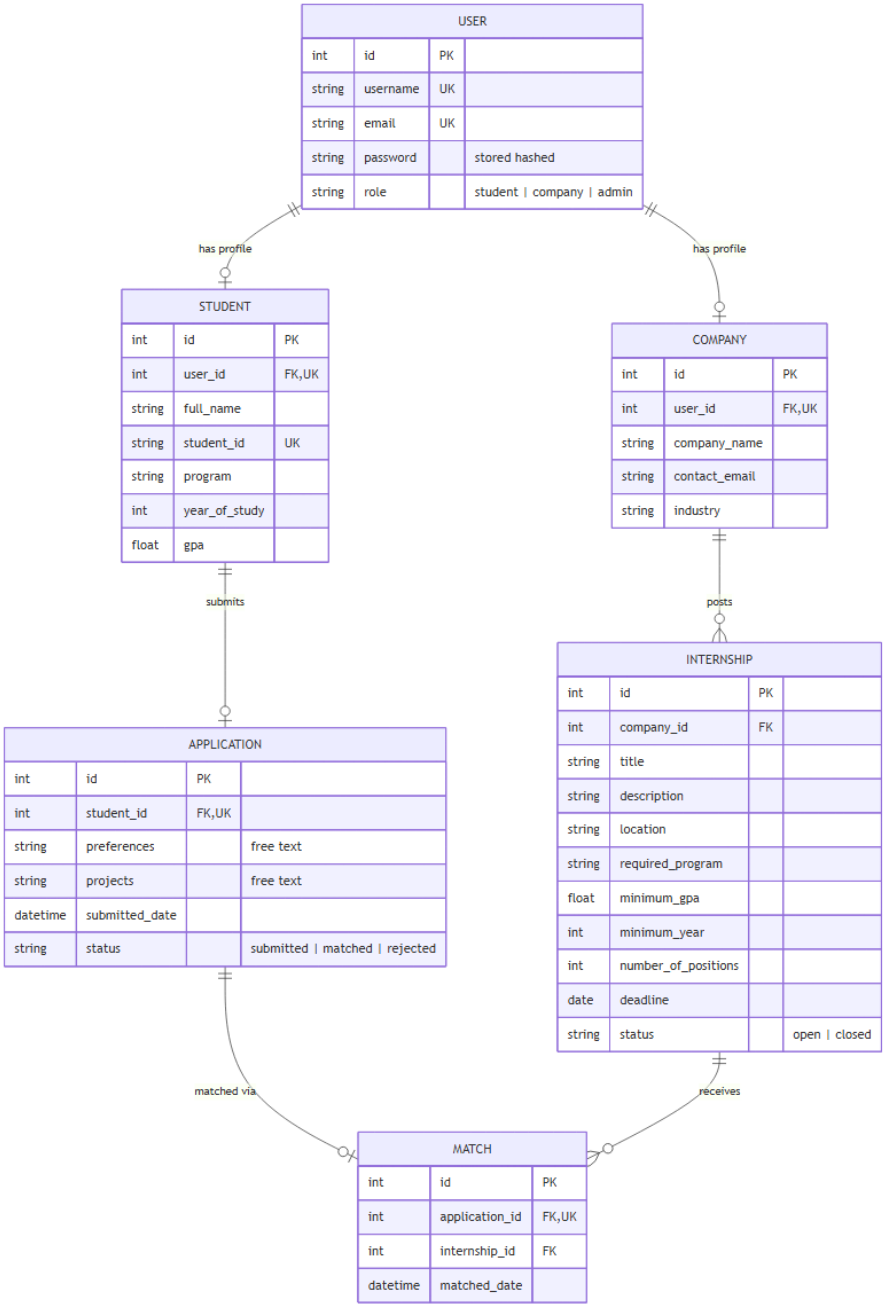
• **Student:** Register with University Email, Log In, Submit Application, View Application Status.
• **Company:** Log In, Publish/Manage Internships (including viewing matched students).
• **Admin:** Log In, Create Company Account, Match Application to Internship («include» Check Requirements; «extend» Reject Application when no internship fits).

Phase 2 decisions:

• **Include:** Match Application to Internship «include» Check Requirements. The system always checks the student's GPA, program and year against the internship's requirements before a match.
• **Extend:** Reject Application (No Suitable Internship) «extend» Match Application to Internship. This happens only when no available internship fits the student.
• **General pool:** students do not apply to a specific internship, so there is no Browse Internships use case. Before a match, only the Admin sees an application. The Admin's match is final: companies do not review or reject matched students.
• **Matched students (Company):** a company sees the students matched to its internships as part of Publish/Manage Internships, not as a separate use case.
• **Gap filled, View Application Status (Student):** this is how a student learns whether they were matched or rejected.
• **Accounts:** students register with a university email address. Companies cannot register themselves; the Admin creates each company account once the company is approved (Create Company Account). Log In is shared by Student, Company and Admin.

Model diagram

First draft. Update this section in Phase 5 when polish revises the model, and note what changed.



Phase 3 decisions (entities and properties named by the student):

• **User -> Student / Company:** each Student or Company profile belongs to exactly one User (user_id is unique). Admins are Users with role = admin and have no profile.
• **Application -> Match (one-to-one, FK on Match):** the Admin's match is final, so an application has at most one Match (application_id is unique).
• **Internship -> Match (one-to-many, FK on Match):** an internship can receive several matches, up to its number of positions.
• **Projects** (added by the student after the first draft) is a free-text property on Application, where the student lists relevant projects.
• **Preferences** are free text. Students do not pick or rank internships (general pool), so Application connects to Internship only through Match.
• **Final match:** the Admin's match is final, so an application has at most one Match and Match has no status.
• **Positions filled:** when the number of Matches reaches number_of_positions, the internship

switches to closed automatically. The Admin can match applications only to open internships.
• **Delete vs close:** a company cannot delete an internship that has any Match rows, because those rows are students' final placements. It closes the internship instead. A closed internship gets no new matches, and existing matches are kept. An internship with no Match rows can be deleted normally. *(Superseded in Phase 4: delete is dropped; companies only close.)*

Assumptions (not asked; revisit in Phase 4/5):

• Every Match counts toward number_of_positions, since matches are final.
• ~~Student -> Application is one-to-many~~. Replaced in Phase 4: one application per student.
• Application.status: submitted until the Admin acts, matched once the Admin matches it (final), rejected when the Admin finds no suitable internship.

Revision (Review Application removed): dropped MATCH.status and the accepted value on APPLICATION.status; removed company accept/reject and re-matching. Application -> Match changed from one-to-many to one-to-one (MATCH.application_id unique).

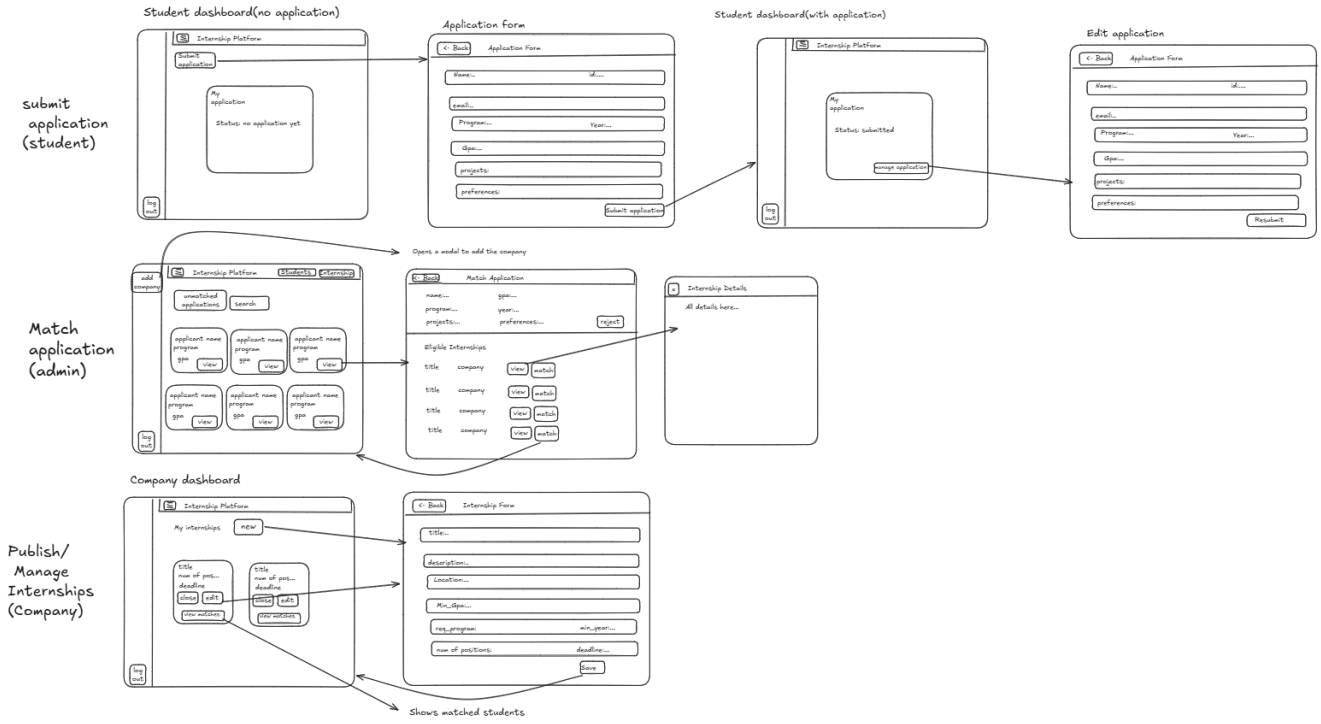
Revision (Phase 4, from the wireframes; accepted by the student):

• **Removed APPLICATION.cover_statement**: it is not on the Application form or the Edit application screen.
• **Student -> Application is one-to-one** (APPLICATION.student_id unique): the student dashboard shows a single "My application" card.
• **Delete internship dropped**: the company card shows only close / edit / view matches, so companies close internships and never delete them.

Wireframes

One student-drawn sheet covers all three workflows (rows: Submit Application, Match Application, Publish/Manage Internships).

Submit Application (Student) · Match Application to Internship (Admin) · Publish/Manage Internships (Company)



Phase 4 workflow decisions

• **Submit Application (Student):** the dashboard shows one "My application" card. The Application form creates the student's STUDENT profile on first submit (name, ID, program, year, GPA), and Edit / Resubmit update that same profile. APPLICATION keeps preferences, projects, date and status. The email field is filled in from USER.email and is read-only (assumed).
• **Resubmit after rejection:** when the status is rejected, "manage application" appears again. The student edits and resubmits, the status goes back to submitted, and the application returns to the Admin's unmatched list. While the status is submitted, the student can still edit (assumed). Once it is matched, the application is locked.
• **View Application Status (Student):** when the status is matched, the card shows the internship title, company name and location (via MATCH -> INTERNSHIP -> COMPANY).
• **Match Application to Internship (Admin):** the Students / Internship header buttons switch the admin dashboard between application cards (unmatched filter + search) and an internship list. Match Application lists only eligible internships (Check Requirements); **match** returns the Admin to the dashboard.
• **Reject Application (Admin):** the **reject** button on Match Application sets the status to rejected and returns the Admin to the dashboard (assumed).
• **Create Company Account (Admin):** the "add company" button in the admin sidebar opens a modal. Its fields are not drawn, so it uses the ERD fields: USER username / email / password (role company) plus COMPANY company_name / contact_email / industry.
• **Publish/Manage Internships (Company):** new / edit open the Internship Form and Save returns to the dashboard. Close sets the status to closed. View matches shows the matched students.
• **Log In / Register with University Email:** these reuse the starter login and register screens, themed in Phase 5. They have no wireframe.

Theming

Branding preferences and how they were applied (landing / login / register).

Preferences (from the student):

• **Colors:** navy (#1b2a4a) for headings, the sidebar and the wordmark. An emerald accent (#047857, hover #065f46, soft tint #ecfdf5) for primary buttons (white text), links, focus rings and the active sidebar link. White (#ffffff) surfaces on a light grey background (#f3f5f8).
• **Type:** Inter (Google Fonts), a clean sans-serif used everywhere, including headings.
• **Tone:** friendly and academic. Example copy: "Find the internship that fits you", "Welcome back! Sign in to your dashboard."
• **Wordmark: InternSearch**, a text wordmark with "Intern" in navy and "Search" in emerald (light emerald #6ee7b7 on the navy sidebar).

How the theme was applied:

• **Tokens:** app/static/css/app.css holds the --brand-* tokens. It also overrides the Bootstrap primary color, .btn-primary, focus rings and cards, so stock components use the brand without extra classes.
• **Landing (/):** white page with a navy and emerald hero and a "How InternSearch works" panel (students apply -> companies post internships -> the department matches). The footer notes that company accounts are created by the department.
• **Login / Register:** white card with an emerald top border on a light grey background, and the wordmark. Register labels the email field "University email" and adds a hint. The domain check itself is not built yet; it comes with the Register with University Email use case.
• **Authenticated shell:** navy sidebar with the wordmark, emerald active link, and a

"Dashboard" link that goes to /admin for admins and /app for everyone else. The 401 page uses the navy -> emerald gradient.
• **Polish (student requests):** the accent went from teal to gold, then from gold to **emerald**. The deeper emerald (#047857) keeps white button text and small text on white readable.
• **Removed starter demo UI:** the public /api/users JSON endpoint (app/routers/users.py) and the landing page's demo-login footer. Placeholder copy on the dashboards was reworded. /config and the starter auth are kept.

Implementation notes

One named workflow at a time. Include verify notes and polish / model revisions (Phase 5). Do not treat the first build as final.

1. Submit Application (Student): verified

• **Route shape (student's choice):** one form route, GET/POST /app/application, used for both the first submit and Edit / Resubmit. The template switches between "Application form" with **Submit application** and "Edit application" with **Resubmit**.
• **Layers:**
• Student and Application SQLModels: app/models/student.py, app/models/application.py.
• StudentRepository and ApplicationRepository.
• ApplicationForm schema.
• ApplicationService: get_for_user, submit_application.
• Thin route: app/routers/application.py.
• Dashboard card in app.html, form in application_form.html.
• **Rules (service):**
• The first submit creates the STUDENT profile, and later submits update it.
• Each submit sets status = submitted and refreshes submitted_date.
• A matched application is refused (the student picked the Service layer for this rule).
• A student ID already used by another account is refused.
• **Roles:** registration now creates users with role = student, matching the ERD's USER.role. Only students can open the form routes. The seeded bob is a student.
• **Deferred:** the matched card's internship title, company and location will be added with the Match workflow, which introduces INTERNSHIP and MATCH.
• **Verify (student):** ran init + run as bob. The submit -> dashboard (Submitted) -> manage -> resubmit flow works. No mismatches reported yet; revisit in the final polish pass.
• **Student snippets:**
• SQLModel Student: user_id FK + unique, student_id unique, plus the profile fields.
• Thin POST route: calls ApplicationService and handles ApplicationError with a flash message and redirect.

2a. Create Company Account (Admin): verified

Built before Publish/Manage Internships. The student chose to start with the admin side, so test company accounts come from the real modal rather than seed data.

• **UI:** an **Add company** button at the top of the admin sidebar (as on the wireframe) opens a modal. It has the COMPANY fields from the ERD (company name, contact email, industry) and the USER login fields (username, login email, password).
• **Layers:**
• Company SQLModel: app/models/company.py.
• CompanyRepository.
• CompanyAccountForm schema.
• CompanyService.create_company_account.

• Thin route POST /admin/companies in app/routers/admin_home.py, admin-only.
• **Rules (service):**
• A username or login email that is already taken is refused.
• The new USER gets role = company.
• If saving the COMPANY profile fails, the new login is removed, so there is never a company login without a profile.
• **Verify (student):** created a company from the admin modal and signed in as it. It lands on the empty company dashboard, which Publish/Manage Internships fills next.
• **Student snippet:** SQLModel Company: user_id FK + unique, company_name, contact_email, industry.

2. Publish/Manage Internships (Company): verified

• **Business rule (student's choice):** a closed internship is final. Its card keeps only **View matches**. Edit and close are hidden in the UI and also refused by the service.
• **Screens:**
• Company dashboard (company_home.html): **My internships** with **New**. Each card shows the title, matched / positions filled, the deadline and an Open/Closed badge, with Close (confirmed), Edit and View matches buttons.
• Internship Form (internship_form.html): used for both new and edit; Save returns to the dashboard.
• View matches (internship_matches.html): each matched student's name, ID, program, year, GPA, email and match date.
• **Layers:**
• Internship and Match SQLModels. MATCH was added here because View matches reads it; the Match workflow writes it.
• InternshipRepository and MatchRepository (match counts and the matched-student join).
• InternshipForm schema.
• InternshipService.
• Thin routes in app/routers/internships.py, company-only via CompanyDep.
• **Rules (service):**
• Only the owning company can see or change an internship.
• Edit and close are refused on a closed internship.
• A new internship's deadline can't be in the past.
• number_of_positions can't drop below the number already matched.
• Reaching number_of_positions matches closes the internship (Phase 3 rule).
• **Verify (student):** re-created a company, then tested new -> save, edit, close and view matches. Reported working with no mismatches; revisit in the final polish pass.
• **Student snippets:**
• SQLModel Internship: company_id FK, the ERD fields, status defaulting to open.
• Thin create route. The first attempt was missing the success redirect (it returned None); the student fixed it after review.

3. Match Application to Internship (Admin): verified

• **Business rule (student's choice):** Check Requirements excludes internships whose deadline has passed.
• **Screens:**
• Admin dashboard (admin.html): **Students / Internships** header buttons. Students has an **Unmatched applications / All** filter, search (name, student ID or program) and application cards with **View**. Internships lists every internship with its company, requirements, filled/positions, deadline and status.

• Match Application (admin_application.html): the student's details, projects and preferences, and **Reject**. **Eligible internships** each have **View** (Internship Details modal) and **Match** (confirmed, because matches are final). Match and Reject both return to the dashboard.
• Student dashboard: a matched card now shows the internship title, company, location and match date (View Application Status; this was deferred from workflow 1).
• **Layers:**
• Match SQLModel.
• ApplicationRepository.search_with_students.
• InternshipRepository.find_matching (the Check Requirements query) and get_all_with_company.
• MatchRepository.create_match (one commit) and get_placement.
• MatchService.
• Thin routes in app/routers/admin_home.py.
• **Rules (service):**
• Eligible means: status = open, deadline ≥ today, the program matches (ignoring case and spaces), GPA ≥ minimum_gpa, and year ≥ minimum_year.
• Only submitted applications can be matched or rejected.
• Eligibility and remaining positions are checked again on the server before a match.
• The match, application.status = matched, and the auto-close (when the last position is filled) are saved together in one commit.
• **Verify (student):** ran the full chain (admin creates a company -> company publishes -> students apply -> admin matches and rejects). Reported that all three workflows work.
• **Student snippets:**
• SQLModel Match: application_id FK + unique (a final match, one per application), internship_id FK, matched_date default now.
• Thin match route: calls MatchService.match_application and handles MatchError, returning to the Match Application screen.

Polish pass 1 (all workflows)

After all three workflows worked, the Guide listed loose ends against the design and the student chose to fix all four. The student supplied the two business values (the .edu domain rule and the 4.3 GPA cap).

• **Register with University Email (Phase 2 use case, previously unenforced):** student sign-up now only accepts an email whose domain ends in **.edu** (for example name@my.uwi.edu). The rule is in AuthService.register_user, which raises a RegistrationError shown on the form, and the form also checks the format in the browser. The service now checks for a duplicate username or email itself, instead of relying on a database error. Company logins (created by the Admin) are not restricted.
• **GPA cap at 4.3:** the Application form's GPA and the Internship form's minimum GPA must be between 0 and 4.3 (app/utilities/grading.py, GPA_MAX). The services enforce it with a readable message, and the inputs carry max="4.3".
• **Deadline-passed badge:** an internship that is open but past its deadline is shown as **Deadline passed** on the company dashboard and the admin Internships list. This makes the student's "past deadline = not eligible" rule visible.
• **Matched-to on admin cards:** in the admin **All** view, a matched application's card shows the internship title and company it was matched to (MatchRepository.get_placements).
• **Model:** no ERD change. The GPA range and the .edu rule are business rules in the services, not new columns.
• **Verify (student):** checked the polish pass and reported it done. Phase 5 exit gate met: all

three workflows work locally, and the student steered the polish (four fixes plus the .edu and 4.3 values).

Deployed app

Phase 6. Public Render URL (not localhost). Markers open this to mark the three workflows.

https://faststarter-07fs.onrender.com

Source code: https://github.com/Gianmarcos794/Comp3613_A1

• Hosted on Render's free plan in Oregon: web service faststarter and Postgres 16 faststarter-db, created with the Render MCP to match render.yaml.
• Start command: python manage.py init --no-drop && python manage.py run --host 0.0.0.0 --port $PORT. Tables are not dropped on reboot, and the seed users are added only if they are missing.
• The free web service sleeps when idle, so the first request can take about a minute. The free database expires on **2026-11-11**.
• **Verify (student):** the database was reset once with a one-off deploy that dropped the tables. The student then ran all three workflows on the public URL (Submit Application as bob, Publish/Manage Internships as comp1, Match Application as admin) and reported that they work.

Logins

Every account a marker needs, including extra users you added. Starter accounts:

• bob / bobpass: student (seeded; Submit Application)
• admin / adminpass: admin (Match Application, Add company)
• comp1 / comppass: company (Publish/Manage Internships). Not seeded: created on the live site with the admin's Add company.

YouTube URL

https://youtu.be/RPX4wAlBhw8

Agent

Claude Code (VS Code extension), with the student-build and student-judge skills; Render MCP for deployment.

Session transcripts

Filled when the Guide builds the report: the agent writes chat markdown into docs/transcripts/; python manage.py report packages them.

Guide packaged **9** chat(s) in docs/transcripts/ (and docs/transcripts.zip).

Index: docs/transcripts/INDEX.md

• 01-setup
• 02-phase1
• 03-phase2
• 04-phase3
• 05-phase4
• 06-phase5
• 07-phase5-wrapup-render-setup
• 08-phase6
• 09-phase6-deploy

Competency (student-judge)

Filled by Guide from the student-judge run when this report was built.

Judged at: 2026-10-12T18:25Z **Evidence pass:** re-read docs/transcripts/01-09 (Claude Code sessions, Phases 1-6) + current docs/report.md, docs/diagrams/, docs/wireframes/, app/routers/ (prior judge.md ignored; none existed)

Student / session: Gian Suarez Nápoles (816042658), Internship Platform **Artifact:** Guide chats for this project, written as docs/transcripts/*.md (Claude Code, VS Code) **Phases in evidence:** 1-6 (COMP 3613; Phase 5 polish, Phase 6 deploy; never generic 0-5)

Totals

Item	Count / value
Metrics on rubric	12 (M1-M12)
N/A (excluded)	0
Metrics scored	12
Scoreable max	48
Awarded total	40 / 48
Overall (avg of scored)	3.3 / 4
Impression mark	17 / 20 (no single Guide confidence logged; round(3.33 / 4 × 20))

Scorecard

ID	Metric	Score / 4	Avg	Evidence
M1	Phase discipline	3	yes	One design phase per chat (02-05), no app code before the wireframe covered every use case, Phase 5 built one workflow at a time with a polish pass before Phase 6. Later scope edits were made by going back to the Phase 2/3 chats.

ID	Metric	Score / 4	Avg	Evidence
M2	Problem framing	4	yes	Framed the domain in their own words: "in this design the student does not apply to a specific internship, they apply and the system after review their application if accepted matches them with internships"; added "view application status" when asked how students learn the outcome; companies are created by the admin "so only trusted companies can post internships".
M3	Decision ownership	4	yes	Named every entity and property themselves (Phase 3 list), owned relationship and rule choices ("an application can only have one active match at a time"), and cut scope deliberately: "admins match is final", then "change to one to one, the company should be able to see its matched students, but as part of publish/manage".
M4	Artefact-before-code	3	yes	Wireframe updated to cover the missing Create Company Account and Reject paths before any code ("check now"). Phase 4 model edits applied ("apply the 3"). Phase 5 built from the ERD and wireframe with no substitute design.
M5	Verification habit	2	yes	Ran each workflow and reported real failures (pasted ASGI traceback; "the dshboard is empty"), but most verify notes are bare confirmations - "they work", "done go to the next one" - with no observed-vs-wireframe detail; Guide had to flag this. Live deploy checked: "the 3 workflows work".
M6	Assignment fit	3	yes	Layered build: thin routes call services (no select/db.exec in app/routers/), rules in services (matched-lock, deadline exclusion, .edu sign-up, GPA cap), starter auth and seed reused, themed (navy + emerald, Inter, InternSearch), deployed on Render.

ID	Metric	Score / 4	Avg	Evidence
M7	Slice explanation	3	yes	Completed SQLModel and thin-route snippets for every workflow (code-check blocks, architecture_ok: yes); picked the Service layer for the matched-lock rule; one route snippet was passed: partial (missing success redirect) and fixed on the second pass.
M8	Prompt quality	3	yes	Phase-tagged openers, entity list in the requested shape, short steering after artefacts existed ("start with the submit aplication", "try emerald instead"). Phase 1 workflows were not in Feature (user) format at first and were tidied by Guide.
M9	Response to pushback	3	yes	Fixed the partial route snippet, iterated the theme (teal -> gold -> emerald), answered every Phase 4 gap question, and took on all four polish items, supplying the .edu and 4.3 values. Polish items were Guide-proposed rather than self-found.
M10	Integrity	4	yes	No suspicion spiral, no paste-back or "just apply this"; python manage.py skills-verify -> "Skill integrity: pass".
M11	Provenance continuity	4	yes	The general-pool and admin-final-match decisions from Phase 2 flow unchanged through the ERD (Application to Match one-to-one, unique student_id), the wireframe and the shipped services.
M12	Sincerity trajectory	4	yes	No sincerity blocks were needed in any chat; suspicion never opened.

Strengths

• Strong Phase 2 ownership: include (Check Requirements), conditional extends, and a new View Application Status use case all came from the student's own descriptions of the flow.
• Student-named entities and properties, with clear relationship and business-rule answers; scope later simplified on purpose (admin match is final) and the model updated to match.
• Wireframe revised to close real coverage gaps (Create Company Account, Reject) before any code.

• Every workflow has student-completed model and thin-route snippets that respect the layers, and no router does persistence inline.
• Full Phase 6: Render Postgres + web service, live URL and all three marker logins in the report, workflows confirmed on the live site.

Gaps (priority order)

1. **Verify notes are thin.** Most Phase 5 confirmations are "they work" / "done" with no comparison against the wireframe; the polish list came from Guide rather than from mismatches the student found.
2. **Deploy care.** After the one-off database reset, the start command was left without --no-drop until Guide flagged it; it is fixed now, but it would have wiped data on every wake.

Phase gate status

Phase	Status	Note
1	met	Internship Platform; workflows named by the student (now three after a deliberate cut)
2	met	Use-case PNG with include/extend from student answers; revised after "admins match is final"
3	met	Student-named entities; ERD with student-chosen relationships and rules
4	met	Wireframe covers all use cases after one update; model edits applied
5	met	Theme applied, three workflows plus Create Company Account built and verified, polish pass 1 applied
6	met	https://faststarter-07fs.onrender.com live on Render Postgres; bob, admin, comp1 logins in report

Recommended next practice

• For one workflow, write a verify note that walks the wireframe screen by screen ("expected X, saw Y") and names at least one mismatch you would change yourself, before asking for the next step.

Integrity note

• Clean

Provenance flags

• None

Sincerity log summary

• Blocks found: 0 | max round: 0 | min/mean/final confidence: n/a | trend: n/a | cleared: n/a

(never entered)

Skips

• Skips: 0/3 used.

Skill integrity

Course skills are hashed at export and compared to .agents/skills.lock.json. Do not edit .agents/skills/, .cursor/skills/, or AGENTS.md.

• Status: **pass**
• Root: e84cd692d0b85eefe546385661958c27d07e8be6c5176a82012f68ccff5c8beb
• none

Full Guide chats written by the agent at report build time.

Setup (Claude Code)

Source: Claude Code (VS Code) session dedbb749-c19a-4677-883c-486d5582a3de. Started: 2026-10-05 16:47 UTC.

Student - 2026-10-05 16:47

can i delete the previous chats?

Guide

tool: Bash - List docs and transcripts folders

I wouldn't delete them yet. You have no saved copies right now: there's no docs/transcripts/ folder in the repo.

Those chats are graded evidence for this assignment. When you build the report, every Guide chat gets copied into docs/transcripts/*.md. The judge then uses them to score you on things like your include/extend answers, your pushback on relationships, how you clarified the wireframes, your code snippets and your polish work. The report script packages whatever is in that folder. It can't pull chats out of the IDE by itself. If you delete a chat now, that evidence is lost and your score could drop.

What I'd do:

1. **Keep them for now**, at least until you've run the report export (python manage.py report --name "..." --id "...").
2. **If you want to clean up sooner**, export each chat first. Copy or export it to a markdown file like docs/transcripts/phase1.md or phase2.md, and commit it. After that, deleting the chat in the IDE is safe.
3. **Once the transcripts are saved and committed**, you can delete the chats.

If you tell me which IDE or tool the earlier chats were in (Cursor, Copilot or OpenCode), I can help you get them into docs/transcripts/.

Student - 2026-10-05 16:47

in claude code i meant

Guide

tool: Bash - List saved Claude Code sessions for this project

Yes. Claude Code keeps each chat as a file on your computer, and deleting a chat means deleting its file. You have two earlier chats for this project, stored in C:\Users\suare\.claude\projects\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\:

Chat file	Last updated
33bf5e85-....jsonl (plus its folder)	Oct 5, 12:46
95c1aa0e-....jsonl (plus its folder)	Oct 5, 12:47
dedbb749-....jsonl	this chat; don't delete it

To delete one, remove its .jsonl file and the folder with the same name. Leave the memory folder alone.

For this assignment, I'd save them first. These are your Guide chats, and the report has to include them as docs/transcripts/*.md. The judge scores your work from them. Right now no copies exist in the repo.

Even if you don't delete anything, Claude Code automatically removes old chats after 30 days by default (the cleanupPeriodDays setting). So copying them into the repo soon is a good idea either way.

I can turn both earlier chats into docs/transcripts/*.md now, and then delete the original files if you still want them gone. Should I do that?

Student - 2026-10-05 16:48

but im starting again so i dont need those

Guide

tool: Bash - Delete the two previous chat sessions

I deleted both earlier chats and their folders. What's left is this chat and the memory folder.

The old chat list in the VS Code panel may keep showing them until you reload the window.

Since you're starting over, check docs/report.md, docs/diagrams/ and docs/wireframes/ too. If those have work from your earlier attempt, you'll probably want to clear or reset them. I haven't touched them.

When you're ready, open a **new chat** for Phase 1 and give it your project and the workflows you want to build, written as Feature (user).

Student - 2026-10-05 16:48

i mean these

Guide

Those are the two chats I already deleted. "Use-case diagram phase 2" and "Assignment start with build skill" were their files, and both are gone from disk. The list still shows them because the panel hasn't refreshed.

To clear them:

1. Press **Ctrl+Shift+P**, then run **Developer: Reload Window**.
2. Open the sessions list again. Only this chat ("Delete previous chats") should be left.

If they still show up after the reload, try opening one. It should come up empty or fail to load. Tell me if it doesn't, and I'll look for a cached copy somewhere else.

Phase 1: project and workflows

Source: Claude Code (VS Code) session bb707d48-39ab-47a3-b422-0c4ee66ec423. Started: 2026-10-05 16:49 UTC.

Student - 2026-10-05 16:49

start the assignment using the build skill

Guide

I'll load the course skill files first.

tool: Bash - Read student-build skill and phases doc

tool: Read - C:\Users\suare\.claude\projects\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\bb707d48-39ab-47a3-b422-0c4ee66ec423\tool-results\bp3otnz4v.txt

Now let me check where the project stands and the assignment briefs.

tool: Bash - Read report, wireframes, and assignment brief

student-build is loaded, and you're at **Phase 1**: docs/report.md is still the blank template and docs/wireframes/ is empty.

How this works:

• **Each chat covers one design phase.** When Phase 1 is done, I'll write it into docs/report.md, stop, and give you the prompt to start Phase 2 in a **new chat**. Phase 5 (theme -> build -> polish) can stay in one chat for all workflows, and Phase 6 is deployment, which comes after polish.
• **You have 3 skips.** If a question feels too hard or pointless, say **skip**. I'll make a stated assumption and move on. A skip can't stand in for your project choice, your workflow names or your wireframes.
• **No app code until Phase 5.**

For this phase, base confidence is below 0.40 because nothing is written yet, so I can ask at most 8 questions.

Question 1: Which project were you assigned on the myeLearning spreadsheet?

1. **Alumni App:** a CRM for staying connected with graduates (profiles, events, networking)
2. **Internship Platform:** student applications matched to company internship positions
3. **MyAdvisor:** degree progress, semester course planning, advisor approval
4. **Research Platform:** poster and presentation submissions, review, and status through the event lifecycle
5. **Student Awards:** volunteer hours, a leaderboard, milestones and prize redemption
6. **Student Accommodation:** an Airbnb-like marketplace with listings, reviews, and tenant and booking management

You can also list your workflows now if you have them. Use at least three lines in this format:

```
Feature (user)
```

For example: Explore/Search Publications (Public). You only need names here. The screen steps go in your Phase 4 wireframes.

Student - 2026-10-05 16:50

2

Guide

Internship Platform it is. That's one question used (1/8).

Question 2: What are your workflows for the Internship Platform?

Name at least three, one per line, in this format:

```
Feature (user)
```

For example: Explore/Search Publications (Public). Each user in brackets is the person or role who does that workflow. Three is the minimum, and you can name more. Names are enough here: the steps go in your Phase 4 wireframes.

Student - 2026-10-05 16:52

publish\manage internships(company) submit application(student) match application to internship(admin) review application(company)

Guide

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

I recorded your four workflows in docs/report.md under **Assigned project** and the workflows section. I only tidied the capitalisation and the slash:

1. Publish/Manage Internships (Company)
2. Submit Application (Student)
3. Match Application to Internship (Admin)
4. Review Application (Company)

Each one already says who does it, so Phase 1 needs nothing more.

Phase 1 done
Artefact: docs/report.md
Base confidence: 0.30
Questions: 2/8
Suspicion: none
Skips: 0/3 used
Report: updated

Your progress is saved in docs/report.md, so the next chat can pick it up from there. **Open a new chat** and paste this to start Phase 2:

Use the student-build skill. Phase 2. Here is my Phase 1 artefact in docs/report.md. Draft the use-case diagram from it.

Phase 2: use-case diagram

Source: Claude Code (VS Code) session d39db66a-d0e4-400e-b59b-7f5bf9b76d31. Started: 2026-10-05 16:52 UTC.

Student - 2026-10-05 16:52

Use the student-build skill. Phase 2. Here is my Phase 1 artefact in docs/report.md. Draft the use-case diagram from it.

Guide

I'll load the course skill and your Phase 1 artefact first.

tool: Bash - List skill/docs files and read the student-build skill

tool: Bash - Read the report and existing use-case JSON

tool: Read - C:\Users\suare\.claude\projects\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\d39db66a-d0e4-400e-b59b-7f5bf9b76d31\tool-results\bg7npvp3g.txt

tool: Bash - Read Phase 2 rules and the Internship Platform brief

student-build loaded: Phase 2, the use-case diagram.

• You have **3 skips** for questions you think are needless. Skips: 0/3 used.

• This chat covers Phase 2 only. Once the diagram is drafted, open a **new chat** for Phase 3.
• The Phase 1 artefact has the project (Internship Platform) and four workflows with three actors: Company, Student and Admin.

Base confidence is **0.65**, so I can ask at most **6** questions before I draft. I'll ask them one at a time.

Question 1 of 6:

Here are your four use cases:

1. Publish/Manage Internships (Company)
2. Submit Application (Student)
3. Match Application to Internship (Admin)
4. Review Application (Company)

Should any of these **«include»** or **«extend»** another, or contain a sub-step that should, instead of standing alone with its own actor line? For each pair you pick, say whether the step is **always** required («include») or **only sometimes** («extend»).

Student - 2026-10-05 16:55

match application should include check requirements cause the system always checks the student's GPA, program and year against the internship's requirements before making a match. review application

Guide

Noted: **Match Application to Internship «include» Check Requirements.** The system always checks GPA, program and year against the internship's requirements before it makes a match.

Your message stops at "review application", so I don't have the rest of that answer. What did you want for **Review Application (Company)**? Does it «include» or «extend» something, or does it relate to another use case such as Match Application? Please say whether that step happens **always** or **only sometimes**.

I'm still counting this as **question

Student - 2026-10-05 16:55

[Request interrupted by user]

Student - 2026-10-05 16:56

match application should include check requirements cause the system always checks the student's GPA, program and year against the internship's requirements before making a match.

Guide

Recorded: **Match Application to Internship «include» Check Requirements** (always runs). Review Application stays as its own use case with no include or extend. Questions: 1/6.

Question 2 of 6:

You named **Submit Application (Student)**, but nothing that lets a student see which internships exist. How does a student find the posting they're applying to, and check its details and requirements, before they submit?

Student - 2026-10-05 16:58

in this design the student does not apply to a specific internship, they apply and the system after review their application if accepted matches them with internships

Guide

Recorded: students apply to a **general pool**, not to a specific posting, so the diagram doesn't need a Browse Internships use case. Questions: 2/6.

Question 3 of 6:

That leaves an ordering question between two of your use cases. **Review Application** belongs to the Company, and **Match Application to Internship** belongs to the Admin. If a student's application isn't aimed at any one company, which company reviews it, and does that review happen **before** the Admin matches it or **after**, once it has been matched to that company's internship?

Student - 2026-10-05 17:00

The company reviews it only after the admin has matched it. Until a match is made, the application belongs to no company and only the admin can see it once the admin matches a student to an internship, that application shows up in review application for the company that posted it, and the company decides whether to accept or reject the student

Guide

Recorded: the order is **Submit -> Admin Match (includes Check Requirements) -> Company Review (accept/reject)**. Before a match, only the Admin can see an application, and after the match it belongs to the company that posted the internship. Questions: 3/6.

Question 4 of 6:

What happens when the company **rejects** a matched student? Does the application end there, or does it go back to the Admin to be matched to a different internship? If it goes back, does that happen **every time** or **only sometimes**, for example only while there are other internships the student still qualifies for?

Student - 2026-10-05 17:04

If a company rejects the student, the application goes back to the admin. If there are still internships available and the student is suitable for one, the admin matches the application to that new company. If not, the student is rejected.

Guide

Recorded. A rejection is **conditional**, so I'll draw it as an «extend» on Review Application that sends the application back to the Admin. If no other suitable internship is left, Match Application ends in the student being rejected, which is also an «extend». Questions: 4/6.

Question 5 of 6:

Under this design the student submits once and then the application moves through Admin match, Company review, and possibly a re-match. None of your use cases shows the student any of that. How does a student find out whether they were matched, accepted or rejected?

Student - 2026-10-05 17:05

The student finds out through a new use case view application status

Guide

Recorded: a new use case, **View Application Status (Student)**. Questions: 5/6.

Question 6 of 6 (the last before I draft):

All three actors work in protected flows. The starter app lets anyone register, but a company that posts internships is a trusted party. How does a **Company** account come to exist? Does the company register itself, or does the Admin create or approve it? And do students simply register and log in?

Student - 2026-10-05 17:07

Students register with their university email so only enrolled students can create accounts. Companies cannot register themselves. The admin creates each company account after the company is approved to take part, so only trusted companies can post internships

Guide

That's all six questions. I'll check how the renderer reads includes/extends and actor sides before I write the JSON.

tool: Bash - Find the use-case renderer and its JSON handling

tool: Bash - Inspect renderer handling of extends, includes and sides

tool: Bash - Check default actor linking for use cases

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\diagrams\use-case.json_

tool: Bash - Render the use-case diagram PNG

The command picked up an app package from a different project (info2602-fastapi-l4). I'll check where that path comes from.

tool: Bash - Check Python path and virtualenv setup

The global Python has an editable install of another project that takes over the name app. I'll use this project's venv instead.

tool: Bash - Render the diagram with the project virtualenv

_tool: Read - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\diagrams\use-case.png_

The PNG rendered and it reads correctly. Next I'll add the embed and notes to the report.

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

I've drafted the use-case diagram from your Phase 1 workflows and your six answers. It's saved as use-case.png, built from use-case.json, and embedded in report.md with notes on each decision.

What's on the diagram:

• **Student:** Register with University Email, Log In, Submit Application, View Application Status.
• **Company:** Log In, Publish/Manage Internships, Review Application.
• **Admin:** Log In, Create Company Account, Match Application to Internship.
• **«include»:** Match Application to Internship always includes Check Requirements (GPA, program and year).
• **«extend»:** Return Application to Admin extends Review Application, and only happens when the company rejects the student.
• **«extend»:** Reject Application (No Suitable Internship) extends Match Application to Internship, and only happens when no internship fits.

Two things I decided without asking:

• **Log In is one use case shared by all three actors.** The alternative was a separate login line from every protected use case, which would clutter the diagram.
• **Student is drawn across the top, not on the left.** I asked for Student on the left and Company and Admin on the right, but the renderer moved Student. Company and Admin still end up on opposite sides, and the diagram reads fine.

python manage.py run won't work with the plain python command in this project. Your global Python has an editable install from another project that also uses the name app, so it loads the wrong code. I ran the course command with ./venv/Scripts/python.exe manage.py usecase instead, so activate venv before you use manage.py.

Phase 2 done
Artefact: docs/report.md (+ docs/diagrams/use-case.json, docs/diagrams/use-case.png)
Base confidence: 0.65
Questions: 6/6
Suspicion: none
Skips: 0/3 used
Report: updated

Your progress is saved in docs/report.md, not just in this chat. Open a **new chat** and paste this, along with your list of entities and their properties:

Use the student-build skill. Phase 3. Here are my entities and properties. Draft the model diagram from docs/report.md.

Student - 2026-10-12 02:46

can you remove the review application pne?

Guide

_tool: Read - c:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

Guide asked: Review Application is your 4th named workflow, and your Phase 3 model relies on it: the company accepts or rejects each Match, and a rejection sends the application back to the Admin. If it's removed, who decides whether a matched student is accepted?

Student - 2026-10-12 02:46

(answer) The user doesn't want to proceed with this tool use. The tool use was rejected (eg. if it was a file edit, the new_string was NOT written to the file). STOP what you are doing and wait for the user to tell you how to proceed.

Student - 2026-10-12 02:46

[Request interrupted by user for tool use]

Student - 2026-10-12 03:00

admins match is final

Guide

I'll take Review Application out of the workflows, the diagram and the model notes.

tool: Bash - Remove Review Application from the diagram JSON and re-render

Now the report: workflows list, Phase 2 notes, and the model parts that relied on company review.

tool: Bash - Update report to remove Review Application and its model effects

_tool: Read -

C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\diagrams\use-case.png_

I removed Review Application, and the Admin's match is now final. The re-rendered use-case.png has no Review Application and no Return Application to Admin. Match Application still includes Check Requirements, and Reject Application (No Suitable Internship) still extends it.

Changes in report.md:

• **Workflows:** there are now three, which is still the minimum. A one-line note says why the fourth was dropped.
• **Phase 2 notes:** I removed the Return Application extend line. The notes now say companies don't review or reject matches, and View Application Status shows matched or rejected.
• **Model:** I removed MATCH.status and the accepted value of APPLICATION.status. These model edits follow from your decision; you didn't name them yourself.
• **Model notes:** the Phase 3 notes about company rejection and re-matching are gone. Every match now counts toward number_of_positions, and a company can't delete an internship that has matches because those are students' final placements. A revision note records all of this.

Two things are still open, and I haven't changed either:

• **Application -> Match is still drawn one-to-many.** Since a match is final, an application can only ever have one. Do you want it changed to one-to-one, with a unique application_id on Match?
• **The Company has no way to see which students were matched to its internships.** Publish/Manage Internships is now its only workflow. Is that intended, or should the company be able to see its matched students?

Skips: 0/3 used.

Student - 2026-10-12 03:03

yes cahnge to one to one, the company should be able to see its matched students, but as part of publish/manage

Guide

tool: Bash - Make Application-Match one-to-one and note company's matched-students view

I made both changes in report.md:

• **One-to-one:** the model diagram now shows Application to Match as APPLICATION ||--o| MATCH, and MATCH.application_id is marked FK, UK (a foreign key that must be unique). I rewrote the relationship note to match, and the revision note records the change from one-to-many.
• **Matched students:** a new Phase 2 note says a company sees the students matched to its internships as part of Publish/Manage Internships. You said it belongs inside that use case, so I didn't add a separate one, and the use-case diagram stays as it is.

Skips: 0/3 used. Your design phases now go up to the model. When you're ready, open a **new chat** for Phase 4 with your wireframe images in docs/wireframes/:

Use the student-build skill. Phase 4. Wireframe images are in docs/wireframes/.

When you draw Publish/Manage Internships, include where the company sees its matched students, because Phase 4 checks that each use case is shown in an image.

Phase 3: model diagram

Source: Claude Code (VS Code) session fcd78062-4fdb-4e37-8a85-ed56ceae84f5. Started: 2026-10-05 17:08 UTC.

Student - 2026-10-05 17:08

Use the student-build skill. Phase 3. Here are my entities and properties. Draft the model diagram from docs/report.md.

Guide

tool: Bash - Read skill files and report

tool: Read - C:\Users\suare\.claude\projects\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\fcd78062-4fdb-4e37-8a85-ed56ceae84f5\tool-results\bt4wzipu8.txt

I'm using **student-build** for **Phase 3, the model diagram**. A few rules for this chat:

• You have **3 skips** for questions that seem too hard or pointless. When you skip one, I'll make a reasonable assumption and keep going.
• This chat covers **only Phase 3**. Phase 4 starts in a new chat.
• Skipping can't be used for the entity list. You have to name the entities and their properties yourself.

I read docs/report.md. Phase 1 and Phase 2 are there: the Internship Platform, four workflows, and the use-case diagram with its include/extend decisions. Your entities and properties aren't in the report or in your message yet, so I'll start there. After that I'll ask about relationships and business rules, and then draft the erDiagram.

Question 1: Which entities does the model need, and which properties does each one have?

Type them in this shape:

```
Internship
- title
- ...
Student
- ...
```

Student - 2026-10-05 17:56

```
User
- username
- email
- password
- role (student, company, admin)

Student
- user
- full name
- student id
- program
- year of study
- gpa

Company
- user
- company name
- contact email
- industry

Internship
- company
```

- title
- description
- location
- required program
- minimum gpa
- minimum year
- number of positions
- deadline
- status (open, closed)

Application
- student
- preferences
- cover statement
- submitted date
- status (submitted, matched, accepted, rejected)

Match
- application
- internship
- matched date
- status (pending, accepted, rejected)

Student - 2026-10-05 17:56

[Request interrupted by user]

Student - 2026-10-05 17:57

User
- username
- email
- password
- role (student, company, admin)

Student
- user
- full name
- student id
- program
- year of study
- gpa

Company
- user
- company name
- contact email
- industry

Internship
- company
- title
- description
- location
- required program
- minimum gpa
- minimum year
- number of positions
- deadline
- status (open, closed)

Application
- student
- preferences
- cover statement
- submitted date
- status (submitted, matched, accepted, rejected)

Match
- application
- internship
- matched date
- status (pending, accepted, rejected)

Guide

Thanks, that list works. I'll use your entities and properties as written. Next come the relationships and business rules. These are the questions left before I draft (up to 5 more).

In Phase 2 you decided that a company can reject a student, and then **Return Application to Admin** lets the Admin re-match the same application to a different internship.

Question 2: How are Application and Match related: one-to-one, or one-to-many (one application can have several Match rows over time)? Which of the two holds the foreign key?

Student - 2026-10-05 18:00

Application and Match are one-to-many. One application can have several Match rows over time, one for each internship the admin matches it to, because a rejected application goes back to the admin and can be matched again. Match holds the foreign key, since each Match row stores the application it belongs to.

Guide

Good. One-to-many with the FK on Match fits re-matching, and it also keeps a history of earlier rejections.

Question 3: On Application, what is preferences? Is it free text (for example, "prefer software roles in Port of Spain"), or does the student rank specific internships? Ranking specific internships would mean a relationship between Application and Internship.

Student - 2026-10-05 18:01

Preferences is free text. The student writes what they are looking for

Guide

That works with the general pool from Phase 2. Students don't pick internships, so there's no link between Application and Internship except through Match.

Question 4: Can one application have more than one Match with status pending (or accepted) at the same time? For example, could the Admin match the same student to two internships at once? If not, what should happen when the Admin tries?

Student - 2026-10-05 18:03

No an application can only have one active match at a time, meaning only one match with status pending or accepted. The admin can only create a new Match after the current one has been rejected

Guide

That's a clear rule. Match will have at most one row per application with status pending or accepted, and new matches are allowed only after a rejection.

Question 5: An Internship has number of positions. Suppose it has 2 positions and already has 2 accepted Matches. Can the Admin still match another application to it? Should the internship switch to closed automatically, and do pending Matches count against the positions?

Student - 2026-10-05 18:04

if the position has already been filled, it should be switched to closed

Guide

Okay. When the number of accepted Matches reaches number of positions, the internship switches to closed, and the Admin can only match to open internships. You didn't say whether

pending Matches count. I'll assume only **accepted** ones count, so pending Matches don't hold a position.

Question 6 (the last one): Publish/Manage Internships lets a company edit or remove its internship. What should happen if a company tries to delete an internship that already has Match rows, especially a pending or accepted one?

Student - 2026-10-05 18:07

A company cannot delete an internship that has any match rows because that would break the history the admin uses for re-matching. Instead, the company can close it by changing its status to closed. A closed internship stops getting new matches, but any pending match still stays in review aplication so the company can accept or reject that student, and accepted matches are kept as they are. Internships with no match rows can be deleted normally.

Guide

_tool: Edit - c:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

Phase 3 is done. I added the model diagram (a Mermaid erDiagram) and your decisions to the **Model diagram** section of docs/report.md. It covers all six of your entities and their properties.

Relationships in the model:

• **User -> Student and User -> Company:** each profile belongs to exactly one User. Admins are Users with the admin role and no profile.
• **Company -> Internship:** one-to-many.
• **Student -> Application:** one-to-many.
• **Application -> Match and Internship -> Match:** both one-to-many, with the foreign keys on Match. This keeps the re-matching history.

Business rules recorded:

• An application can have only one active Match (pending or accepted) at a time.
• An internship closes automatically when its positions are filled.
• An internship with Match rows can only be closed, not deleted.

Assumptions to check in Phase 4/5 (I didn't ask about these, so they're also listed in the report):

• Only accepted Matches count toward number_of_positions. You didn't say whether pending ones do.
• A student can have more than one Application over time.
• Application.status follows the current Match. When a company rejects, it goes back to submitted. It becomes rejected only when the Admin finds no suitable internship.

Phase 3 done
Artefact: docs/report.md (Model diagram + Phase 3 decisions)
Base confidence: 0.65
Questions: 6/6
Suspicion: none
Skips: 0/3 used
Report: updated

Your progress is saved in docs/report.md, not only in this chat. Next, draw your wireframes and put the images in docs/wireframes/. Then open a **new chat** and paste:

Use the student-build skill. Phase 4. Wireframe images are in docs/wireframes/.

Student - 2026-10-12 02:35

can you add projects to the application model

Guide

_tool: Edit - c:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

_tool: Edit - c:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

I added projects to the Application entity in the model diagram in docs/report.md. I also added a line to the Phase 3 decisions noting that you added it after the first draft.

I'd already asked all 6 questions allowed for this phase, so I didn't ask what form projects should take. I treated it as **free text**, the same as preferences: the student writes about their relevant projects in one field.

If you want each project stored separately with its own details (a title, description or link, for example), it would need to be its own Project entity, with one Application having many Projects. Tell me which properties a project should have and I'll redraw the diagram that way.

Phase 3 is otherwise unchanged. Next, put your wireframe images in docs/wireframes/, open a new chat and paste:

```
Use the student-build skill. Phase 4. Wireframe images are in docs/wireframes/.
```

Phase 4: wireframes

Source: Claude Code (VS Code) session ce5f2d70-e638-45e6-8e93-000aff809da9. Started: 2026-10-12 03:33 UTC.

Student - 2026-10-12 03:33

Use the student-build skill. Phase 4. Wireframe images are in docs/wireframes/.

Guide

I'll start by loading the course skill files, the current report, and the wireframes.

tool: Bash - List skill/docs files and read skill instructions

_tool: Read - c:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

tool: Read - C:\Users\suare\.claude\projects\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\ce5f2d70-e638-45e6-8e93-000aff809da9\tool-results\bb3r9gf43.txt

_tool: Read - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\wireframes\wireframes.png_

tool: Bash - Read use-case diagram source

student-build, Phase 4. You have **3 skips**. This chat covers Phase 4 only; Phase 5 starts in a new chat.

I checked docs/wireframes/wireframes.png against the Phase 2 use cases and the Phase 3 ERD.

Coverage

Use case	Image	Covered
Submit Application (Student)	Student dashboard (no application) -> Application form -> Student dashboard (with application) -> Edit application	yes
View Application Status (Student)	"My application / Status" card on the student dashboard	yes
Match Application to Internship (Admin)	Admin dashboard -> Match Application -> Internship Details	yes
Check Requirements	"Eligible Internships" list on Match Application	yes
Publish/Manage Internships (Company)	Company dashboard -> Internship Form, plus close / edit / view matches	yes
Log In, Register with University Email	No image. I'm assuming these reuse the starter login and register screens and get themed in Phase 5.	yes (starter)
Create Company Account (Admin)	No image	no
Reject Application (No Suitable Internship)	No image, and the Match Application screen has no reject action	no

Create Company Account has no wireframe. Companies can't register themselves, so without this screen no Company user can log in, and Publish/Manage Internships can't start. Please add an image for it to docs/wireframes/. I'll wait for that before moving on to the model suggestions.

While you do that, one question about the other gap:

On the Match Application screen, what does the Admin do when none of the eligible internships fit (or the list is empty)? Where do they reject the application, and what happens next?

You can answer that in words, or show it in the new image if it's easier to draw.

Student - 2026-10-12 03:42

check now

Guide

tool: Bash - List wireframe files

Same single file, updated recently. Re-reading it.

_tool: Read - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\wireframes\wireframes.png_

The updated image covers both gaps:

• **Create Company Account:** the "add company" button in the admin sidebar opens a modal. The modal's fields aren't drawn, so I'll use the ERD fields: USER username, email and password with role company, plus COMPANY company_name, contact_email and industry. You don't need to redraw it.
• **Reject Application:** the Match Application screen now has a **reject** button. I'll assume it sets APPLICATION.status = rejected, sends the Admin back to the unmatched list, and shows "Status: rejected" on the student's dashboard.

Every use case now has an image. Log In and Register reuse the starter screens.

Model suggestion so far

• APPLICATION.cover_statement: remove it? It isn't on the Application form or the Edit application screen.

Design notes (nothing to redraw)

• The admin header has **Students** and **Internship** buttons, but no image shows where they go and no use case needs them. In Phase 5 we'll either drop them or point them at simple lists.
• The company card has only **close** and **edit**. The Phase 3 rule said an internship with no matches can be deleted, but no delete action is drawn.

Next question (2/5): The student dashboard shows a single "My application" card, and the Submit button disappears once an application exists. Does a student get only one application, or can they submit a new one later, for example after being rejected? Phase 3 assumed one student could have many applications, so your answer decides whether APPLICATION.student_id becomes unique.

Student - 2026-10-12 03:44

those buttons change what you see in the dashboard, in tha tone youre seeing the students if you click internship it shows the internships

Guide

Got it. The **Students** / **Internship** buttons switch the admin dashboard between two views:

• **Students:** the application cards with the "unmatched applications" filter and search, as drawn.
• **Internship:** a list of internships. No image shows it, so in Phase 5 I'll reuse the company card layout (title, number of positions, deadline) and add the company name and open/closed status. Both are already in the ERD.

That's a dashboard toggle, not an extra screen, so the note is resolved.

Question 2/5 is still open: the student dashboard shows one "My application" card, and the Submit button disappears once an application exists. Is it **one application per student**, or can a student submit a **new one later**, for example after being rejected?

Student - 2026-10-12 03:46

keep one application per student If it's rejected manage application shows up again, the student edits it, clicks resubmit, and the status goes back to Submitted so the admin sees it again

Guide

Recorded:

• **Student -> Application becomes one-to-one.** APPLICATION.student_id is unique.
• **Resubmit after rejection:** when the status is rejected, the "manage application" button shows again. Resubmitting sets the status back to submitted, and the application returns to the Admin's unmatched list.
• **My assumption:** while the status is submitted the student can still edit through manage application, and resubmitting doesn't change the status. Once the status is matched, the application is locked because the match is final.

Question 3/5: The dashboard card only shows "Status: ...". Once the status is **matched**, what does the student see? Does the card show which internship and company they were placed with, or only the word "matched"?

Student - 2026-10-12 03:47

When the status is matched, the card shows the internship title, the company name, and the location, so the student knows where they were placed

Guide

That doesn't need new fields. Title and location come from MATCH -> INTERNSHIP, and the company name comes from INTERNSHIP -> COMPANY. I'll note it as a display rule for the matched status.

Question 4/5: The Application form asks for Name, ID, Email, Program, Year and GPA, but in the ERD those live on STUDENT (and email on USER), not on APPLICATION. Register only collects the university email and password. Which of these do you want?

• **A.** The Application form creates and updates the student's STUDENT profile. Editing GPA on resubmit changes the profile itself.
• **B.** Copy those values onto APPLICATION as a snapshot, so the Admin sees what was submitted and the profile stays separate.
• **C.** Something else.

Student - 2026-10-12 03:47

a

Guide

Going with **A**:

• Submitting the Application form for the first time **creates** the student's STUDENT row (full_name, student_id, program, year_of_study, gpa). Edit and resubmit **update** that same row. APPLICATION keeps only preferences, projects, submitted_date and status.
• **My assumption:** the form's email field is filled in from USER.email and can't be edited, because it's the university email the student registered with.

Question 5/5, last one. Which of these model revisions should I apply? Pick any or all:

1. Remove APPLICATION.cover_statement. It isn't on any form.
2. Make APPLICATION.student_id unique, so Student -> Application is one-to-one, per your answer.
3. Drop **delete internship**, so companies only **close**, matching the wireframe card.
Otherwise I keep the Phase 3 rule and add a delete action in Phase 5 for internships with no matches.

Student - 2026-10-12 03:48

apply the 3

Guide

Applying all three to docs/report.md, and embedding the wireframe with the coverage blocks.

tool: Bash - Apply Phase 4 model revisions and embed wireframes in report

Phase 4 is done. All three model changes are applied, and the wireframe is embedded in docs/report.md with a coverage note for each use case.

Model changes

• **Cover statement removed:** APPLICATION.cover_statement is gone, since it isn't on any form.
• **One application per student:** APPLICATION.student_id is now unique, and the diagram shows STUDENT ||--o| APPLICATION.
• **Close only:** companies can no longer delete an internship. The old Phase 3 delete rule is marked as replaced.

Workflow decisions written to the report

• **Application form and profile:** the form creates the student's profile on the first submit, and editing or resubmitting updates that same profile.
• **Rejected applications:** a rejected application can be edited and resubmitted. It goes back to "submitted" and returns to the Admin's unmatched list.
• **Matched applications:** once matched, the application is locked, and the card shows the internship title, company name and location.
• **Admin dashboard:** the Students and Internship buttons switch between the application cards and an internship list.
• **Reject and add company:** the reject button and the "add company" pop-up form are included as drawn.

What I assumed rather than asked (also noted in the report):

• The email field on the form is filled in from the registered email and can't be edited.
• A student can still edit while the status is "submitted".
• Reject sends the Admin back to the dashboard.
• The "add company" form uses the account and company fields from the model.
• Log In and Register use the starter screens, which get themed in Phase 5.

Phase 4 done
Artefact: docs/report.md (+ docs/wireframes/wireframes.png)
Base confidence: 0.75
Questions: 5/5
Suspicion: none
Skips: 0/3 used
Report: updated

Your progress is saved in docs/report.md, not only in this chat. Open a **new chat** and paste

this, filling in your preferences:

```
Use the student-build skill. Phase 5. Wireframes are in docs/wireframes/. Here are theming and branding preferences: ...
```

Phase 5: theming, build, polish

Source: Claude Code (VS Code) session 05e9bf5d-79fa-4ee8-8b08-274870f999f9. Started: 2026-10-12 03:49 UTC.

Student - 2026-10-12 03:49

Use the student-build skill. Phase 5. Wireframes are in docs/wireframes/. Here are theming and branding preferences: ...

Guide

tool: Bash - Read build skill, phases, report, and app layout

tool: Read - C:\Users\suare\.claude\projects\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\05e9bf5d-79fa-4ee8-8b08-274870f999f9\tool-results\b2dsmoz40.txt

I'm using the student-build skill for Phase 5: theming, then building, then polishing. You have **3 skips** for questions that seem needless. Phase 5 can stay in this chat for every workflow. Phase 6 (deploy) only comes after polish.

I read docs/report.md and docs/wireframes/wireframes.png. The ERD, the three workflows and the wireframe coverage are all in place, so Phase 5 can start.

Your prompt still has the template's **"..."** where the branding preferences go, so I have nothing to theme from yet. What branding should the Internship Platform use? Please fill in:

```
Phase 5. Theming and branding.
Colors: ...   (e.g. a primary + accent, or "navy and gold", or hex codes)
Type: ...     (e.g. a Google Font, or "clean sans-serif", "serif headings")
Tone: ...     (e.g. formal/academic, friendly, corporate)
Logo or wordmark: ...   (e.g. a text wordmark like "InternMatch", or an image you'll put in app/static/img/)
```

Once you answer, I will:

1. Write the theming into docs/report.md.
2. Apply it as CSS tokens in app/static/css/app.css and restyle the landing page (/), the login page and the register page. The authenticated shell will use the same tokens.
3. Remove the starter demo pages that don't belong in this app, keeping /config and auth.

Then you run python manage.py run and tell me what you see before we start the first workflow (Publish/Manage Internships, Submit Application, or Match Application, whichever you choose).

Student - 2026-10-12 04:24

Colors: navy, teal accent, white and light grey backgrounds type: inter from google fonts, clean sans-serif for everything tone: friendly/academic logo or wordmark: InternSearch

Guide

tool: Bash - Read templates, CSS listing, and routers

tool: Bash - Read CSS, 401 page, and config router

Applying the theme now: tokens, landing page, login, register and the app shell. I'll also

remove the starter demo bits.

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\static\css\app.css_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\landing.html_

tool: Bash - Find references to the demo users API

tool: Bash - Remove demo users API and switch font to Inter

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\login.html_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\register.html_

Now the authenticated shell, the placeholder home pages and the 401 page.

tool: Bash - Apply brand tokens to shell, home pages, and 401

tool: Bash - Check seeded admin role value

Good. Now I'll record the theming in the report.

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

The InternSearch theme is in place on the landing page, login, register and the logged-in shell. I haven't run the app, so please check it yourself.

What changed:

• **Colors and font:** navy for headings and the sidebar, teal for buttons and links, white cards on light grey, and Inter everywhere. All of it comes from the --brand-* tokens in app.css, and stock Bootstrap buttons pick up the teal automatically. The Inter font is loaded in base.html.
• **Landing page:** a hero ("Find the internship that fits you") next to a three-step panel: students apply, companies post internships, the department matches. The footer says company accounts are created by the department. See landing.html.
• **Login and register:** a white card with a teal top border and the wordmark. Register now labels the field "University email". It doesn't check the domain yet; that comes with the register workflow. See login.html and register.html.
• **Logged-in shell:** navy sidebar with the InternSearch wordmark and a Dashboard link that sends admins to /admin and everyone else to /app. The 401 page is restyled to match. See authenticated-base.html.
• **Removed:** the public /api/users demo endpoint, which listed every user and their email, and the demo-login footer on the landing page. /config and the starter login are untouched.
• **Report:** the colors, font, tone, wordmark and the changes above are under Theming in docs/report.md.

Please check it yourself:

```
python manage.py run
```

Open /, /login and /register, then sign in as bob / bobpass and admin / adminpass to see both dashboards. What did you see? Is anything off from what you wanted, such as how dark the navy is, how bright the teal is, or the wording? We'll adjust the theme before starting the first workflow.

Student - 2026-10-12 04:28

can you change the teal for gold

Guide

tool: Bash - Locate all teal/accent usages

Gold is too light to read as white-on-gold buttons or as small gold text on white. So I'm using a bright gold fill with navy text on buttons, and a deeper gold for links and small text.

tool: Bash - Swap teal accent for gold with legible variants

Now updating the report.

tool: Bash - Update report theming to gold accent

I swapped the teal for gold everywhere; there's no teal left in the CSS or templates. I haven't run the app, so check it with python manage.py run.

Gold is too light for white text or small text on white, so I used three shades:

• **Buttons and the active sidebar link:** gold #d4a017 with **navy text**. It turns a darker gold #b8890f on hover.
• **Wordmark "Search" and the hero highlight:** the darker gold #b8890f, so they're readable on white.
• **Links, small labels and icons:** a deep gold #8a6508.
• **Sidebar wordmark:** a light gold #f5c84c on the navy.

The focus rings, the card top border, the 401 gradient and the light-gold kicker background use gold too. All of it is in app.css and authenticated-base.html. The Theming section of docs/report.md now has the gold colors and notes the change from teal as a polish step you asked for.

Restart the app (or refresh) and look at /, /login, /register and a dashboard. Does the gold look right to you, or should it be warmer or more muted? Once you're happy with the theme, which workflow do you want to build first: **Publish/Manage Internships**, **Submit Application**, or **Match Application to Internship**?

Student - 2026-10-12 04:29

try emerald instead

Guide

tool: Bash - Swap gold accent for emerald and update report

I've switched the accent to emerald everywhere; there's no gold left in the CSS or templates. I haven't run the app, so check it with python manage.py run.

• **Buttons and the active sidebar link:** emerald #047857 with white text, darker #065f46 on hover. I used this deeper emerald because a brighter one is hard to read with white text.
• **Wordmark "Search", the hero highlight, links and icons:** the same emerald.
• **Kicker and step-icon backgrounds:** a soft emerald tint #ecfdf5.
• **Sidebar wordmark:** a light emerald #6ee7b7 on the navy.
• **Focus rings, card top border and 401 gradient:** emerald as well.

The changes are in app.css and authenticated-base.html. The Theming section of docs/report.md now lists the emerald colors and records the teal -> gold -> emerald changes as polish you asked for.

Refresh /, /login, /register and a dashboard. Does the emerald look right with the navy? If it does, which workflow should we build first: **Publish/Manage Internships**, **Submit Application**, or **Match Application to Internship**?

Student - 2026-10-12 04:30

start with the submit aplication

Guide

_tool: Read - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\wireframes\wireframes.png_

tool: Bash - Read models, repos, services, deps, database

Guide asked: Your wireframe has two form screens: Application form (Submit application) and Edit application (Resubmit). They show the same fields. How should the routes be shaped?

Student - 2026-10-12 04:31

(answer) Your questions have been answered: "Your wireframe has two form screens: Application form (Submit application) and Edit application (Resubmit). They show the same fields. How should the routes be shaped?"="One form route". You can now continue with these answers in mind.

Guide

I'll build it as **one form route**: GET/POST /app/application with one template that shows empty or saved fields and labels the button Submit or Resubmit.

Here's a quick check on the next piece:

Guide asked: A student whose application is already matched sends the form again (for example from an old browser tab). Where should that resubmission be turned away?

Student - 2026-10-12 04:32

(answer) Your questions have been answered: "A student whose application is already matched sends the form again (for example from an old browser tab). Where should that resubmission be turned away?"="Service". You can now continue with these answers in mind.

Guide

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\models\student.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\models\application.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\models__init__.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\repositories\student.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\repositories\application.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\repositories__init__.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\schemas\application.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\services\application_service.py_

Next: the student role, the routes and the templates.

tool: Bash - Add StudentDep and make registration create students

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\application.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\user_home.py_

tool: Bash - Check how seeding picks the role

Role is passed explicitly, so the CLI seed works. I'll align the /config reseed too, then write the templates.

tool: Bash - Align config reseed role and register router

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\app.html_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\application_form.html_

tool: Bash - Check the app still imports

Python picked up the app package from a different project (info2602-fastapi-l4). I'll retry with this repo first on the path.

tool: Bash - Retry import with this repo on the path

tool: Bash - Inspect manage.py and Python path setup

This repo has its own venv. The global Python has another project's app package installed as editable, and that copy shadows this one. I'll use the repo's venv instead:

tool: Bash - Check app imports with the repo venv

Everything around your two snippets is scaffolded and the app imports. It won't work until you fill in the two gaps below.

Your turn: two snippets

1. SQLModel: open app/models/student.py and complete the marked STUDENT SNIPPET region. Add the remaining STUDENT columns from the erDiagram in docs/report.md, with the keys and constraints the ERD shows.

2. Route: open app/routers/application.py and complete the marked ROUTE SNIPPET in submit_application_action. After a successful Submit application or Resubmit, follow the wireframe back to the dashboard. If the submission is refused, show the reason and return to the form. Keep the route thin: bind the request and call a service. No select() or db.exec() in the route. Everything you need is already imported at the top of the file.

What I built:

• **Application model:** app/models/application.py, with one application per student.
• **Repositories:** StudentRepository and ApplicationRepository.
• **Form schema:** ApplicationForm.
• **ApplicationService:** in application_service.py.
• It creates the STUDENT profile on the first submit and updates it on edits.
• A resubmit sets the status back to submitted.
• A matched application is refused, which was your Service pick.
• So is a student ID that another account already uses.

• **Routes and templates:**
• GET /app/application loads the form.
• The /app dashboard has the "My application" card with the status, Submit application, and Manage application.
• application_form.html is the form. The email field is read-only and the button says Submit application or Resubmit.
• **Roles:** new sign-ups now get role = "student", as the ERD says, and the form routes require that role. bob is seeded as a student.

Two things to know:

• **Use the repo's venv.** Your global Python has another project's app package installed, so python manage.py run there loads the wrong app. Run venv\Scripts\activate first.
• **Re-seed once:** run python manage.py init. This creates the new tables and makes bob a student. It drops existing data.
• **Not built yet:** for a matched application, the card will show the internship title, company and location once the Match workflow adds the Internship and Match tables.

When both snippets are done, tell me. I'll review them. Then you run the app, sign in as bob / bobpass, submit, then manage and resubmit, and tell me what you saw compared with the wireframe.

Student - 2026-10-12 04:36

```
ERROR:    Exception in ASGI application
Traceback (most recent call last):
  File
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\uvicorn\protocols\http\httptools
_impl.py", line 422, in run_asgi
    result = await app(  # type: ignore[func-returns-value]
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        self.scope, self.receive, self.send
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\uvicorn\middleware\proxy_head
ers.py", line 63, in __call__
    return await self.app(scope, receive, send)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\applications.py",
line 1163, in __call__
    await super().__call__(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\applications.py",
line 96, in __call__
    await self.middleware_stack(scope, receive, send)
  File
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\middleware\errors.py",
line 186, in __call__
    raise exc
  File
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\middleware\errors.py",
line 164, in __call__
    await self.app(scope, receive, _send)
  File
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\middleware\base.py",
line 197, in __call__
    response = await self.dispatch_func(request, call_next)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\main.py", line 43, in recover_uninitialized_db
    return await call_next(request)
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\middleware\base.py",
line 173, in call_next
    raise app_exc from app_exc.__cause__ or app_exc.__context__
  File
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\middleware\base.py",
line 149, in coro
    await self.app(scope, receive_or_disconnect, send_no_error)
  File
```

```
"C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\middleware\sessions.py", line 91, in __call__
    await self.app(scope, receive, send_wrapper)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\middleware\exceptions.py", line 63, in __call__
    await wrap_app_handling_exceptions(self.app, conn)(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\_exception_handler.py", line 53, in wrapped_app
    raise exc
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\_exception_handler.py", line 42, in wrapped_app
    await app(scope, receive, sender)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\middleware\asyncexitstack.py", line 18, in __call__
    await self.app(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\routing.py", line 676, in __call__
    await self.middleware_stack(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 2734, in app
    await route.handle(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 1780, in handle
    await self.original_router.handle(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 2789, in handle
    await included_router._handle_selected(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 1800, in _handle_selected
    await original_route.handle(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 1279, in handle
    await app(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 158, in app
    await wrap_app_handling_exceptions(app, request)(scope, receive, send)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\_exception_handler.py", line 53, in wrapped_app
    raise exc
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\starlette\_exception_handler.py", line 42, in wrapped_app
    await app(scope, receive, sender)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 144, in app
    response = await f(request)
               ^^^^^^^^^^^^^^^^
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 706, in app
    raw_response = await run_endpoint_function(
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    ...<3 lines>...
    )
    ^
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\fastapi\routing.py", line 352, in run_endpoint_function
    return await dependant.call(**values)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\user_home.py", line 20, in user_home_view
    student, application = service.get_for_user(user)
                           ~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\services\application_service.py", line 22, in get_for_user
    student = self.student_repo.get_by_user_id(user.id)
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\repositories\student.py", line 13, in get_by_user_id
    return self.db.exec(select(Student).where(Student.user_id == user_id)).one_or_none()
                                              ^^^^^^^^^^^^^^^^^^
  File "C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\venv\Lib\site-packages\pydantic\_internal\_model_construction.py", line 290, in __getattr__
    raise AttributeError(item)
AttributeError: user_id
```

Guide

tool: Bash - Check whether the snippets were filled in

You haven't done anything wrong. This crash is the gap you're meant to fill.

AttributeError: user_id means the Student model has no user_id column yet. I checked both files and the two snippet regions are still at TODO:

• app/models/student.py: only id exists. The dashboard needs Student.user_id to find your profile, so it crashes as soon as bob loads /app.
• app/routers/application.py: the handler body is still ..., so Submit would return nothing.

Next steps:

1. Complete the **STUDENT SNIPPET**. Add the remaining STUDENT columns from the erDiagram in docs/report.md (the STUDENT { ... } block), with the keys and constraints it shows.
2. Complete the **ROUTE SNIPPET** in submit_application_action.
3. **Re-run python manage.py init.** The crashed run already created a student table with only an id column. The app won't add columns to an existing table, so it has to be rebuilt after you finish the model.
4. Run python manage.py run again and sign in as bob / bobpass.

Tell me when both snippets are in. I'll review them before you test the flow. If either one has you stuck, tell me what you've tried and I'll give you a nudge rather than the answer.

Student - 2026-10-12 04:55

is it ok now?

Guide

tool: Bash - Check end of route file and app import

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

I've added the workflow notes and these checks to docs/report.md.

Now check it yourself. Activate the venv first. You must re-init, because the old student table only has id:

```
venv\Scripts\activate
python manage.py init
python manage.py run
```

Sign in as bob / bobpass and go through the flow:

1. The dashboard shows "My application", status **No application yet**, and a **Submit application** button.
2. Fill in the form and submit. You should land back on the dashboard with status **Submitted** and a **Manage application** button.
3. Click Manage application. You should see "Edit application" with your saved values, then **Resubmit**.
4. Optional: register a second student and try to reuse bob's student ID. The form should refuse it with a message.

What did you see compared with the wireframe? Mention anything that looks off or that you'd change: wording, layout, which fields are required, the GPA range, and so on.

Student - 2026-10-12 04:58

ok it works lets go to the next workflow

Guide

tool: Bash - Record verify note for workflow 1

Guide asked: How should we get a company account to test Publish/Manage Internships?

Student - 2026-10-12 04:58

(answer) The user answered: "How should we get a company account to test Publish/Manage Internships?"="lets start with the admin one first". Read the answers carefully - they may request clarification, changes, or that you not proceed - and follow what they actually say.

Guide

tool: Bash - Check current Student model and admin route usage

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\models\company.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\repositories\company.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\schemas\company.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\services\company_service.py_

tool: Bash - Wire company schema, repo lookup, model registry, sidebar slot

Now the admin route and the modal:

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\admin_home.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\admin.html_

tool: Bash - Check app imports after company scaffolding

I've scaffolded the Admin's **add company** modal, and the app imports cleanly. It won't work until you complete the snippet below.

Your turn: one snippet

Open app/models/company.py and complete the marked COMPANY SNIPPET region. Add the remaining COMPANY columns from the erDiagram in docs/report.md, with the keys and constraints the ERD shows.

What I built around it:

• **Sidebar button:** an **Add company** button at the top of the admin sidebar, where your wireframe draws it. It opens a modal with:
• the COMPANY fields from the ERD: company name, contact email, industry.
• the login fields: username, login email, password.
• These fields weren't drawn in the wireframe, so I used the ERD fields as Phase 4 recorded.
• **Route:** POST /admin/companies in admin_home.py. It's a thin, admin-only route that calls the service and shows a flash message.
• **Service:** CompanyService.create_company_account in company_service.py.
• It refuses a username or login email that's already taken.
• It creates the USER with role = "company", then the COMPANY profile.

• If saving the profile fails, it removes the new login, so you never get a company login with no company behind it.
• **Supporting code:** CompanyRepository, the CompanyAccountForm schema, CompanyUserCreate (role company), and UserRepository.get_by_email.

When the snippet is done:

```
python manage.py init
python manage.py run
```

Run init again so the new company table gets your columns. Then sign in as admin / adminpass, click **Add company**, create a company, and sign out. Sign in with the new company login. For now it lands on a placeholder dashboard; the Publish/Manage Internships workflow will fill it.

Tell me when the snippet is in, and what you saw.

Student - 2026-10-12 05:01

done, check

Guide

tool: Bash - Verify import and log company notes in report

The Company snippet passes review. It matches the COMPANY block in the ERD:

• user_id is a foreign key to user.id and unique=True, so each company login has exactly one profile.
• company_name, contact_email and industry are there.
• The app imports cleanly.

I've logged this piece in docs/report.md under Implementation notes, as "2a. Create Company Account".

Now check it yourself:

```
python manage.py init
python manage.py run
```

1. Sign in as admin / adminpass and click **Add company** in the sidebar. Create a company, for example username acme with password acmepass.
2. Try creating it again with the same username. You should get a red "already taken" message.
3. Sign out and sign in as the new company. It lands on a placeholder dashboard; the next workflow replaces that.

What did you see? Once the company login works, we'll start **Publish/Manage Internships**, with your Internship model and route snippets.

Student - 2026-10-12 05:14

the dshboard is empty,start now

Guide

Good, the company login works. The empty dashboard is the part this workflow fills. I've marked Create Company Account as verified.

tool: Bash - Mark Create Company Account verified in report

Guide asked: Once a company closes an internship, what can it still do with that card?

Student - 2026-10-12 05:15

(answer) Your questions have been answered: "Once a company closes an internship, what can it still do with that card?"="View matches only". You can now continue with these answers in mind.

Guide

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\models\internship.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\models\match.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\repositories\internship.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\repositories\match.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\schemas\internship.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\services\internship_service.py_

Next: the company-only dependency, the routes and the templates.

tool: Bash - Add CompanyDep and register models, repos, router

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\internships.py_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\user_home.py_

Now the three templates:

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\company_home.html_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\internship_form.html_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\internship_matches.html_

A title containing an apostrophe would break the confirm dialog's JavaScript, so I'm switching it to a fixed message:

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\company_home.html_

tool: Bash - Check app imports after internship scaffolding

Everything around your two snippets is scaffolded, and the app imports cleanly.

Your turn: two snippets

1. SQLModel: open app/models/internship.py and complete the INTERNSHIP SNIPPET. Add the remaining INTERNSHIP columns from the erDiagram in docs/report.md, with the keys, constraints and defaults the ERD and your rules need. Use the ERD's column names exactly; the service maps the form onto them.

2. Route: open app/routers/internships.py and complete the ROUTE SNIPPET in create_internship_action. Save the form, then follow the wireframe: **Save** returns to the company dashboard. If the save is refused, show the reason and return to the Internship Form. Keep the route thin. Everything you need is already imported.

What I built around it:

• **Company dashboard** (company_home.html):
• **My internships** with a **New** button.
• Each card shows the title, matched / positions filled, the deadline and an Open/Closed badge.
• Buttons: **Close** (asks for confirmation), **Edit** and **View matches**. Closed cards show only View matches, as you chose.
• **Internship Form** (internship_form.html): every field from your wireframe, used for both new and edit.
• **View matches** (internship_matches.html): each matched student's name, ID, program, year, GPA, email and match date. It stays empty until the Match workflow runs.
• **InternshipService rules:**
• Only the company that owns an internship can see or change it.
• Edit and close are refused on a closed internship.
• A new internship's deadline can't be in the past.
• Positions can't be lowered below the number already matched.
• Filling every position closes the internship (your Phase 3 rule).
• **Supporting code:**
• Match model, which View matches needs; the Match workflow uses it next.
• InternshipRepository and MatchRepository.
• InternshipForm schema.
• CompanyDep, so only companies can open these routes.

When both snippets are in, tell me and I'll review them. Then you run:

```
python manage.py init
python manage.py run
```

init wipes the data, so recreate your company through the admin's **Add company** first. Then sign in as that company and test new -> save, edit, close, and view matches.

Student - 2026-10-12 05:21

is the route ok?

Guide

_tool: Read - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\internships.py_

The **model passes**. The **route is almost there**: the thin-route structure is right, but the success path is missing.

SQLModel (internship.py): passes.

• company_id is a foreign key to company.id.
• All the ERD columns are there with the right types (deadline: date, minimum_gpa: float).

• status defaults to "open".

Route (internships.py:48-65): partial.

- Thin: it builds the InternshipForm, calls service.create_internship, and has no SQL.
- A refused save (for example a past deadline) shows the reason and goes back to the form.
- **Nothing happens after a successful save.** Once the try succeeds, the function runs on to the ... at line 65 and returns None. The internship is saved, but the company sees a blank page instead of returning to the dashboard as the wireframe shows.

Finish the success path and delete the leftover ... and TODO markers. Look at what your Submit Application route does after a successful save.

Tell me when it's fixed and I'll look again. Then run init and test it.

Student - 2026-10-12 05:22

now?

Guide

tool: Bash - Re-check create route and import

tool: Bash - Log Publish/Manage workflow notes in report

The route passes now. After a successful save it shows "Created ..." and returns to the company dashboard, as in the wireframe. It's still thin, and the app imports cleanly.

The old TODO and ... lines below the return never run, so they're harmless, but you can delete them.

I've logged the workflow, your choice, the snippets and the review round in docs/report.md.

Now check it yourself:

```
python manage.py init
python manage.py run
```

1. Sign in as admin / adminpass and use **Add company** to create a company again, since init wiped it. Sign out.
2. Sign in as the company. You should see the empty "My internships" dashboard with **New**.
3. Click **New**, fill in the form and **Save**. You should land back on the dashboard with an Open card.
4. Try a deadline in the past. It should be refused with a message.
5. **Edit** a card and change something. Then **Close** it: confirm, and only View matches should remain.
6. **View matches** should say no students have been matched yet.

What did you see compared with the wireframe? Note anything you'd change about the card, the form or the wording, and we'll polish it before moving on to **Match Application to Internship (Admin)**.

Student - 2026-10-12 05:25

done go to the next one

Guide

I've marked Publish/Manage Internships as verified. Your last two "done"s didn't mention any changes. That's fine for now, but once all three workflows exist I'll ask you to go through each one and point out anything that doesn't match how you want it to work. The marking looks for that polish step.

tool: Bash - Mark Publish/Manage verified in report

Guide asked: During Check Requirements, should an open internship whose deadline has already passed still be listed as eligible?

Student - 2026-10-12 05:26

(answer) Your questions have been answered: "During Check Requirements, should an open internship whose deadline has already passed still be listed as eligible?"="Exclude past deadline". You can now continue with these answers in mind.

Guide

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\models\match.py_

tool: Bash - Re-read application repo and route header

Now the repositories:

tool: Bash - Add admin listing, eligibility, and match repo methods

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\services\match_service.py_

tool: Bash - Add student lookup and matched placement on student card

Now the admin routes. Your route snippet is the **match** action.

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\routers\admin_home.py_

_tool: Read - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\admin.html_

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\admin.html_

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\admin_application.html_

tool: Bash - Check app imports after match scaffolding

Everything around your two snippets is scaffolded, and the app imports cleanly.

Your turn: two snippets

1. SQLModel: open app/models/match.py and complete the MATCH SNIPPET. Add the MATCH columns from the erDiagram in docs/report.md, with the keys and constraints the ERD and the "final match" rule need. Use the ERD's column names exactly; the repositories and View matches already use them.

2. Route: open app/routers/admin_home.py and complete the ROUTE SNIPPET in match_application_action. Match the application to the chosen internship, then follow the wireframe: **match** returns to the Admin dashboard. If the match is refused, show the reason and return to the Match Application screen. Keep the route thin. There's already a _match_service(db) helper in that file, and MatchError is imported.

What I built around it:

• **Admin dashboard** (admin.html):
• **Students / Internships** header buttons.
• **Students** has an **Unmatched applications / All** filter, search by name, student ID or

program, and application cards (name, status, program, GPA, year, **View**).
• **Internships** lists every internship with its company, requirements, filled/positions, deadline and status.
• **Match Application** (admin_application.html):
• The student's details plus projects and preferences, and a **Reject** button.
• **Eligible internships**, each with **View** (opens an Internship Details modal) and **Match** (asks for confirmation, since matches are final).
• **MatchService rules** (match_service.py):
• **Check Requirements:** an internship is eligible if it's open, its deadline is today or later (your pick), the program matches (ignoring case and spaces), and the student's GPA and year meet the minimums.
• **Match:** only a submitted application can be matched, and the eligibility check runs again on the server.
• **Positions:** a match is refused if no positions are left, and the match that fills the last position closes the internship.
• **One save:** the match, the application status and the internship status are saved in a single commit.
• **Reject:** only a submitted application can be rejected. The student can then edit and resubmit, as already built.
• **Student dashboard:** a matched card now shows the internship title, company, location and match date. This was the item deferred from workflow 1.

When both snippets are in, tell me and I'll review them. Then init and run, and test the whole chain: admin creates a company -> company publishes an internship -> student submits an application -> admin matches it (and rejects another one).

Student - 2026-10-12 05:30

are the snippets ok?

Guide

tool: Bash - Review match route snippet and import

tool: Bash - Log Match workflow notes in report

Yes, both snippets pass, and you got the success path right on the first try this time. The app imports cleanly.

SQLModel (match.py): passes.

• application_id is a foreign key to application.id and unique=True. That's what makes a match final: one match per application.
• internship_id is a foreign key to internship.id and not unique, so an internship can have several matches, up to its positions.
• matched_date defaults to now.

Route (admin_home.py): passes, and it's thin.

• It calls service.match_application.
• A MatchError shows the reason and goes back to the Match Application screen.
• A success shows a message and returns to the dashboard, as the wireframe shows.

As before, the leftover TODO and ... below the return can be deleted.

I've logged this workflow in docs/report.md.

All three workflows are built. Now run the full chain:

```
python manage.py init
python manage.py run
```

1. **Admin** (admin / adminpass): Add company, then sign out.
2. **Company:** publish an internship. Use a required program like Computer Science, a min GPA, a min year, **2 positions** and a future deadline.
3. **Students:** sign in as bob, then register a second student. Submit an application from each with a matching program, GPA and year.
4. **Admin:** the Students view lists both applications. **View** one, open the internship's details, then **Match**. Reject the other.
5. **Check the results:**

• bob's dashboard should show the internship, company and location.
• The rejected student should see Rejected and be able to resubmit.
• The company's **View matches** should list bob.

This is also the **polish pass**. While you click through all three workflows, compare each screen with your wireframe and tell me what you'd change. For example: labels and wording, what each card shows, the search or filter behaviour, anything that feels like one click too many, or a rule you'd like to work differently. "Looks fine" on everything is a valid answer, but at least note what you checked.

Student - 2026-10-12 05:34

they work

Guide

tool: Bash - Mark Match workflow verified in report

Guide asked: Which of these should we polish? (pick any, or write your own under Other)

Student - 2026-10-12 05:36

(answer) The user answered: "Which of these should we polish? (pick any, or write your own under Other)"="can you do all". Read the answers carefully - they may request clarification, changes, or that you not proceed - and follow what they actually say.

Guide

I'll do all four. Two of them depend on facts only you know:

Guide asked: Which email domain should student sign-up require? / What is the maximum GPA on your grading scale?

Student - 2026-10-12 05:37

(answer) The user answered: "Which email domain should student sign-up require?"="it should be an edu domain", "What is the maximum GPA on your grading scale?"="4.3". Read the answers carefully - they may request clarification, changes, or that you not proceed - and follow what they actually say.

Guide

tool: Bash - Re-read auth and service code to edit

tool: Bash - Add .edu sign-up rule and GPA 4.3 cap

One fix: browsers now compile pattern attributes in strict regex mode. In that mode an unescaped - inside [...] makes the pattern invalid, and the browser silently ignores it.

tool: Bash - Fix pattern, add deadline badge and matched-to cards

The sed didn't change the pattern, so I'll fix it directly:

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\templates\register.html_

Now the polish notes in the report:

tool: Bash - Record polish pass in report

All four fixes are in, and the app imports cleanly. I haven't run the app, so check them yourself.

1. University email at sign-up

• The register page only accepts an address ending in .edu, such as name@my.uwi.edu.
• The rule lives in AuthService.register_user. If it's refused, the form shows "Please sign up with your university email...". The browser also checks the format before submitting.
• Company logins created by the Admin aren't restricted.
• The seeded bob@example.com still works because seeding bypasses sign-up.

2. GPA capped at 4.3

• Both the student's GPA and an internship's minimum GPA must be between 0 and 4.3.
• The services enforce it with a readable message, and the inputs carry max="4.3".
• The value lives in one place, grading.py.

3. Deadline-passed badge

• An internship that's open but past its deadline now shows an amber **Deadline passed** badge. It appears on the company dashboard and in the admin Internships list.

4. Matched-to on admin cards

• In the admin **All** view, a matched application's card shows which internship and company it went to.

There were no ERD changes; these are service rules. Everything is written up under "Polish pass 1" in docs/report.md.

Check these yourself. No init is needed, since there are no new tables. Just python manage.py run.

1. Register with a @gmail.com address (refused), then with @my.uwi.edu (accepted).
2. Enter a GPA of 4.5 on an application or internship (refused).
3. Note: the company form won't save a past deadline. To see the badge, edit an existing internship's deadline to yesterday.
4. As admin, switch to **All** and look at a matched card.

What did you see? Is there anything else you'd tune, now that you've clicked through every screen? Any wording, layout or rule counts. When you're satisfied, Phase 5 is done and we can hand off Phase 6 (deploy).

Student - 2026-10-12 05:38

is done

Guide

tool: Bash - Record Phase 5 completion and logins in report

Phase 5 is complete. All three workflows work on your machine, and you steered a polish pass: you chose all four fixes and set the .edu sign-up rule and the 4.3 GPA cap.

Phase 5 done

Artefact: docs/report.md (Theming, Implementation notes 1 / 2a / 2 / 3, Polish pass 1, Logins)
Workflows: Submit Application ✔ · Publish/Manage Internships ✔ (+ Create Company Account) · Match Application ✔
Code checks: 13 logged (choices + SQLModel and thin-route snippets for every workflow; 1 partial route fixed after review)
Polish: theme accent teal -> gold -> emerald; .edu sign-up, GPA ≤ 4.3, deadline-passed badge, matched-to on admin cards
Suspicion: none
Skips: 0/3 used
Report: updated

In docs/report.md I also updated the Logins section:

• bob / bobpass is a student.
• admin / adminpass is the admin.
• There's no seeded company login. After deploy, create one with **Add company** on the live site so markers can test Publish/Manage Internships.

Your progress is in the report, not only in this chat.

Next: Phase 6 (deploy). You can continue here or open a short new chat. Use this prompt:

Use the student-build skill. Phase 6 deploy. Local polish is done. Deploy with the Render MCP; put the public URL and logins in docs/report.md.

Before deploying, commit your work. Nothing from Phase 5 is committed yet: the theme, the three workflows and the polish are all only in your working tree.

Phase 5 wrap-up and Render MCP setup

Source: Claude Code (VS Code) session 083bd170-f46d-44d5-99a7-357555a174d8. Started: 2026-10-12 05:39 UTC.

Student - 2026-10-12 05:39

Use the student-build skill. Phase 6 deploy. Local polish is done. Deploy with the Render MCP; put the public URL and logins in docs/report.md.

Guide

tool: Bash - Read skill files and deploy config

tool: ToolSearch

tool: Bash - Read Phase 6 guidance and report state

tool: ToolSearch

tool: Bash - Check push status and MCP config

I can't deploy yet because the Render MCP isn't connected to this session. I searched the loaded tools twice and found no Render tools, only Higgsfield, Gmail, Drive and Docs. The repo's Render config is in .vscode/mcp.json, but that file is for Copilot. Claude Code doesn't read it, and the repo has no .mcp.json.

Everything else is ready:

• **Phase 5 exit gate is met.** docs/report.md records the student's verify notes and Polish pass 1.
• **main matches origin/main.** Render will build from https://github.com/Gianmarcos794/Comp3613_A1.
• **render.yaml is set up.** It defines a free Postgres database (faststarter-db) and a free web service (faststarter) with /health as the health check, which starts with init --no-drop and then run.

To connect Render (about 2 minutes):

1. Create an API key in the Render Dashboard under Account Settings -> API Keys.
2. In a terminal in this folder, run this with your key in place of <YOUR_KEY>. Don't paste the key into this chat and don't commit it:

``bash claude mcp add --transport http render https://mcp.render.com/mcp --header "Authorization: Bearer <YOUR_KEY>" ``

3. Reload VS Code, or start a new Claude Code session, so the Render tools load.
4. Send the Phase 6 prompt again.

After that, I will:

1. Select your Render workspace.
2. Create the faststarter-db database (Postgres 16, free, Oregon).
3. Create the faststarter web service from your GitHub repo, with the build and start commands from render.yaml. It will get ENV=production, DATABASE_URI from the database's connection string, and generated values for SECRET_KEY and CONFIG_PASSWORD.
4. Watch the deploy logs until /health responds.
5. Fill in **Deployed app** (the public URL) and **Logins** in docs/report.md. The database password stays out of the report.

You'll still need to do two things yourself after it deploys. First, open the URL and log in as admin. Second, use **Add company** to create a company login, since none is seeded, and tell me its username so I can add it to the report.

Skips: 0/3 used

Student - 2026-10-12 14:55

```
s://mcp.render.com/mcp : The term 's://mcp.render.com/mcp' is not recognized as the name of a cmdlet, function,
script file, or operable program. Check the spelling of
the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ s://mcp.render.com/mcp --header "Authorization: Bearer <s://mcp.rende ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (s://mcp.render.com/mcp:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

Guide

The paste lost the start of the command, so PowerShell tried to run s://mcp.render.com/mcp as a command. It also looks like the URL ended up where your API key should go.

Paste this as **one line** in the VS Code terminal, then replace YOUR_KEY with the key from Render (Account Settings -> API Keys). The key starts with rnd_.

```
claude mcp add --transport http render https://mcp.render.com/mcp --header "Authorization: Bearer YOUR_KEY"
```

If claude isn't recognised in that terminal, you can store the key in an environment variable and let me write the config file. Run this, using your real key:

```
setx RENDER_API_KEY "rnd_[REDACTED]"
```

Then tell me, and I'll add a .mcp.json that reads ${RENDER_API_KEY}. That way the key itself never goes into the repo.

Either way:

1. Fully reload VS Code (Ctrl+Shift+P -> "Developer: Reload Window").
2. Send the Phase 6 prompt again.

Don't paste the key into this chat.

Student - 2026-10-12 14:58

'claude' is not recognized as an internal or external command, operable program or batch file.

Guide

_tool: Write - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\.mcp.json_

That's expected: the claude command-line tool isn't installed on your machine, only the VS Code extension. So I've added a .mcp.json to the project instead. It tells Claude Code how to reach Render and reads your API key from an environment variable, so no secret is stored in the repo.

Your steps:

1. In the VS Code terminal, set the key once (put your real key between the quotes):

``powershell setx RENDER_API_KEY "rnd_[REDACTED]" ``

2. **Fully quit VS Code and reopen it.** setx only takes effect in newly started programs, and "Reload Window" isn't enough.
3. If Claude Code asks whether to use the project MCP server **render**, approve it.
4. Send the Phase 6 prompt again:

``text Use the student-build skill. Phase 6 deploy. Local polish is done. Deploy with the Render MCP; put the public URL and logins in docs/report.md. ``

Once the Render tools load, I'll create the database and web service and finish the report.

Phase 6: deploy and transcripts

Source: Claude Code (VS Code) session ae64d6aa-77b8-4816-880e-f8417491feef. Started: 2026-10-12 14:59 UTC.

Student - 2026-10-12 14:59

Use the student-build skill. Phase 6 deploy. Local polish is done. Deploy with the Render MCP; put the public URL and logins in docs/report.md.

Guide

I'll start by loading the course skill files and the deploy config.

tool: Bash - Read Phase 6 guidance, render.yaml, and git remote

Now checking the report's Phase 5 state, whether the code is pushed, and what logins init seeds.

tool: Bash - Check push status, report sections, and seeded users

Phase 5 shows verified workflows plus a polish pass, so the deploy gate is met. Next I'm checking which users get seeded and the report's deploy sections.

tool: Bash - Show seeding code and report deploy sections

Gate met and the code is pushed (main matches origin/main). Moving on to Render: checking the workspace and any existing resources so I don't create duplicates.

tool: ToolSearch

_tool: mcp__render__list_workspaces_

tool: Bash - Check whether the Render API key variable is set

I couldn't deploy: the Render MCP returns unauthorized on every call, so nothing was created on Render and docs/report.md is unchanged.

Cause: .mcp.json sends Bearer ${RENDER_API_KEY}. You have RENDER_API_KEY saved as a Windows user variable, but this VS Code / Claude Code session started before it was set. The session never picked it up, so the MCP sent an empty token.

Fix:

1. Fully quit VS Code (every window, not just this chat) and reopen the project. A Reload Window may not be enough.
2. Start a short new chat with: Use the student-build skill. Phase 6 deploy. Local polish is done. Deploy with the Render MCP; put the public URL and logins in docs/report.md.
3. If it still says unauthorized, the key itself is probably wrong or revoked. Make a new one at Render Dashboard -> Account Settings -> API Keys and save it as RENDER_API_KEY again.

Everything else is ready:

• **Phase 5 is finished.** All three workflows are verified locally and the polish pass is recorded, so Phase 6 is allowed.
• **The code is pushed.** main matches origin/main (https://github.com/Gianmarcos794/Comp3613_A1). Only the new .mcp.json is uncommitted, and Render doesn't need it.

Once the MCP works, the plan follows render.yaml:

• Create a free Postgres 16 database faststarter-db in Oregon.
• Create the free Python web service faststarter from your repo, with the build/start commands and /health check from render.yaml.
• Set DATABASE_URI to the database's internal connection string, and generate SECRET_KEY and CONFIG_PASSWORD.
• Watch the deploy logs, then add the URL to ## Deployed app in docs/report.md. The database password stays out of the report.

Logins: the start command runs init --no-drop, which seeds bob / bobpass (student) and admin / adminpass (admin) on Render too. No company account is seeded. After the deploy, log in as admin on the live site, use **Add company** to create one, and tell me its username and password so I can list it under ## Logins.

Student - 2026-10-12 15:03

i cant developear reload window i need the transcript of the chats

Guide

tool: Bash - Check transcript exporter and stored Claude chats

tool: Bash - Summarize first prompt and date of each saved chat

Most chats map clearly to a phase. Two show no typed prompts, so I'm checking them and the Phase 4/5 chats more closely.

tool: Bash - Inspect the chats with unclear phase

_tool: Write - C:\Users\suare\AppData\Local\Temp\claude\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\ae64d6aa-77b8-4816-880e-f8417491feef\scratchpad\jsonl_to_md.py_

tool: Bash - Generate transcripts and scan them for secrets

tool: Bash - Fix answer detection, regenerate, and spot-check

No secrets survived. The answer fix worked, but the Guide headers still repeat, so I'm checking the merge logic.

tool: Bash - Show merge-related lines in converter

The assistant-side edit never applied, so question answers weren't captured either. Fixing it directly.

_tool: Read - C:\Users\suare\AppData\Local\Temp\claude\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\ae64d6aa-77b8-4816-880e-f8417491feef\scratchpad\jsonl_to_md.py_

tool: Bash - Check which chats used multiple-choice questions

_tool: Edit - C:\Users\suare\AppData\Local\Temp\claude\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\ae64d6aa-77b8-4816-880e-f8417491feef\scratchpad\jsonl_to_md.py_

_tool: Edit - C:\Users\suare\AppData\Local\Temp\claude\c--Users-suare-OneDrive-Documents-GitHub-Comp3613-A1\ae64d6aa-77b8-4816-880e-f8417491feef\scratchpad\jsonl_to_md.py_

tool: Bash - Regenerate transcripts and spot-check answers

Phase 6: Render deploy, reset, and report

Source: Claude Code (VS Code) session ff438e20-acaf-4b85-95cd-10c068c2bde6. Started: 2026-10-12 15:20 UTC.

Student - 2026-10-12 15:20

Use the student-build skill. Phase 6 deploy. Local polish is done. Deploy with the Render MCP; put the public URL and logins in docs/report.md.

Guide

I'll start by loading the student-build skill and phase guidance.

tool: Bash - Load skill and Phase 6 guidance

tool: ToolSearch

Let me check the report for the Phase 5 exit gate, the Phase 6 details in the framework doc, and whether the repo is pushed.

tool: Bash - Read Phase 6 rules, report, and push status

tool: Bash - Read Phase 6 section and report outline

Phase 5 exit gate is met (all three workflows verified, plus a polish pass), and main is pushed. Now the deploy section of the README, the report tail, and how the seed/marker logins work.

tool: Bash - Read report tail, README deploy, seed logins

tool: Bash - Check seed users and DB URI handling

Everything is in place. Now the Render workspace.

_tool: mcp__render__list_workspaces_

Only one workspace exists, so I'll use it. First I'm checking for existing resources so I don't create duplicates.

_tool: mcp__render__list_services_

_tool: mcp__render__list_postgres_instances_

There are no existing resources for this project; the Inventory-App ones are unrelated and I'll leave them alone. Creating the database to match render.yaml:

_tool: mcp__render__create_postgres_

_tool: mcp__render__create_postgres_

The database is being created (free plan, expires 2026-11-11). Next I need its internal connection string.

_tool: mcp__render__get_postgres_

tool: Bash - Check config defaults for deploy env vars

tool: Bash - Generate secret key and config password

_tool: mcp__render__create_web_service_

The web service is created at **https://faststarter-07fs.onrender.com** and its first build is running. I'll put the URL and logins into the report now.

_tool: Read - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

I've created the database and web service on Render, but they aren't connected to each other yet, so the app isn't ready for markers. The Render MCP doesn't give me the database's connection string, and the app needs it in DATABASE_URI. Without it, the app quietly falls back to the SQLite setting in .env.example, and that data would be wiped on every restart.

What I set up (all in your one workspace, "My Workspace"; your existing Inventory-App resources weren't touched):

Resource	Details
Postgres faststarter-db	Free plan, Oregon, Postgres 16, expires 2026-11-11
Web service faststarter	Free plan, Oregon, deploys from main automatically. Build, start command and settings (PYTHON_VERSION, ENV, SECRET_KEY, CONFIG_PASSWORD) match render.yaml
Public URL	https://faststarter-07fs.onrender.com

I generated SECRET_KEY and CONFIG_PASSWORD myself, so they appear in this chat. Leave them out when this chat is copied into docs/transcripts/.

In docs/report.md, the Deployed app section now has the URL, how the app is hosted, and a note that it sleeps when idle and the database expiry date. The report change isn't committed. When you push it, Render will redeploy on its own.

What you need to do:

1. **Connect the database.** In the database's dashboard page, go to **Connect** and copy the **Internal** Database URL. In the web service dashboard, go to **Environment**, add

DATABASE_URI with that value, then save and redeploy. Paste it there rather than here, so the database password doesn't end up in the chat or the transcripts.
2. **Check it yourself.** Once the redeploy is live, open /health and then run all three workflows on the public URL. Log in as bob/bobpass (Submit Application) and admin/adminpass (Match Application).
3. **Create the company login.** Company accounts aren't seeded, so as admin, use **Add company** to create one on the live site. Then tell me its username and password so I can add them under Logins in the report.

Tell me once you've set DATABASE_URI and what you saw. I'll check the deploy logs to confirm the app is using Postgres, and if anything fails I'll look into it.

Student - 2026-10-12 15:27

done that

Guide

_tool: mcp__render__list_deploys_

_tool: mcp__render__list_logs_

tool: Bash - Find the log line that prints the DB URI

The starter already has a mask_database_uri helper, so I'll use it in that log line.

tool: Bash - View init function imports

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\app\cli.py_

Postgres is now connected and the live site is using it. Your redeploy is live, and its log shows Database ready (postgresql://...@dpg-db6fm2brjlhs738be48g-a/faststarter_db_s18z). On that first start, the seed created bob and admin in Postgres.

Problem: that same log line printed the full database URL, password included. The Render logs are only visible on your account, but the password has also come into this chat.

• I changed app/cli.py:44 to mask the password with the starter's existing mask_database_uri, so new logs show user:****@host. It isn't committed yet. When you commit and push it with the report change, Render redeploys automatically. Your data stays, because of --no-drop.
• When this chat goes into docs/transcripts/, leave out the database password, plus the SECRET_KEY and CONFIG_PASSWORD from earlier.
• To be safe, you could also reset the database password in the Render dashboard (if your plan offers credential rotation). If you do, update DATABASE_URI with the new Internal URL.

I still need two things before Phase 6 is finished:

1. **What you saw on the live site.** Did all three workflows work there: Submit Application (bob), Publish/Manage Internships (company) and Match Application (admin)?
2. **The company login.** Give me the username and password of the company you created with admin's **Add company** on the live site. I'll add it to Logins in docs/report.md. If you haven't created one yet, do that first; it's the only way markers can test the company workflow.

Student - 2026-10-12 15:28

comp1 username comppass password

Guide

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

I added the company login to the Logins section of docs/report.md:

• bob / bobpass: student (Submit Application)
• admin / adminpass: admin (Match Application, Add company)
• comp1 / comppass: company (Publish/Manage Internships), created on the live site, not seeded

One thing still open: you haven't said whether all three workflows worked on the live site. Log in as each of the three accounts on https://faststarter-07fs.onrender.com and tell me what you saw. I'll record that in the report as the Phase 6 check, and fix anything that's broken.

Also, two changes are only local so far: the report edits and the app/cli.py change that stops the log printing the database password. Commit and push them; Render will redeploy automatically and your data stays.

Student - 2026-10-12 15:36

can you restart the database?

Guide

I can't do either version of that from here. The Render MCP tools I have can create a database, read it and run read-only queries, but there's no tool to restart, suspend or wipe one. "Restart" could also mean two different things:

1. **Restart the database server** without losing data. That has to be done in the database's dashboard page. If your plan has no restart option there, restarting the web service from its dashboard (**Manual Deploy -> Restart service**) reopens all the app's database connections. That usually fixes whatever a database restart would have.
2. **Reset the data** back to only the seeded users. This wipes everything, including comp1 and any internships or applications you created on the live site. You'd do it once by running python manage.py init (without --no-drop) against the Render database, then recreate comp1 with admin's **Add company**.

Which one do you mean, and what's going wrong on the live site? If something is failing, tell me what you clicked and what you saw, and I'll check the Render logs. A restart may not be the fix.

Student - 2026-10-12 15:36

i need to delete the data

Guide

I can't run the wipe from here. The Render MCP has no tool to drop tables or change a service's start command. Running init from your PC against the database won't work either: it currently accepts no outside connections.

The simplest way is to run one deploy with the start command that drops the tables, then switch it back:

1. **Push first.** Commit and push the app/cli.py change, so the wipe deploy doesn't print the database password in the logs again.
2. Go to the web service settings -> **Settings** -> **Start Command**. Change it to the following (only --no-drop is removed) and save:

``bash python manage.py init && python manage.py run --host 0.0.0.0 --port $PORT ``

3. Run **Manual Deploy -> Deploy latest commit** and wait until it's live. The log should show Dropping all tables..., then create bob and create admin.

4. **Change the start command back right away**:

``bash python manage.py init --no-drop && python manage.py run --host 0.0.0.0 --port $PORT `` If you leave the first one in place, every redeploy and every wake from sleep could wipe the data again, including while markers are testing.

5. Log in as admin and recreate comp1 / comppass with **Add company**. Everything else is deleted, including all internships and applications.

Once you've done that, tell me. I'll check the logs to confirm the tables were dropped and the start command is back to --no-drop, and I'll run a read-only query on the database to confirm only bob, admin and comp1 are left. Then try all three workflows on the live site and tell me what you saw, so I can record it as the Phase 6 check.

Student - 2026-10-12 15:39

the 3 workflows work what now?

Guide

Let me check whether the wipe went through and the start command is back to --no-drop, then record your check.

_tool: mcp__render__get_service_

_tool: mcp__render__query_render_postgres_

_tool: mcp__render__list_logs_

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

Once the start command is fixed and comp1 exists again, the deploy part of Phase 6 is finished. What's left, in order:

1. **Commit and push** the report and app/cli.py changes. Render redeploys on its own, and with --no-drop your data stays.
2. **Video:** record your demo of the three workflows on the live URL and put the link under **YouTube URL** in docs/report.md. Don't show the database password or the Render environment variables on screen.
3. **Export the report:** ask me to build the report. I'll run the judge and write docs/judge.md, then save the Guide chats into docs/transcripts/ with the database password and keys removed. Then I'll run python manage.py report --name "..." --id "..." with your name and student ID, which produces the PDF.

Student - 2026-10-12 18:11

(selected lines 19-19 of report.md)

xhttps://youtu.be/RPX4wAlBhw8 this is the link for the video

Guide

tool: Bash - View YouTube section of report

_tool: Edit - C:\Users\suare\OneDrive\Documents\GitHub\Comp3613_A1\docs\report.md_

I added https://youtu.be/RPX4wAlBhw8 under **YouTube URL** in docs/report.md. I left off the "x" in front of the link, since that looked like a typo. Make sure the video is set to Public or Unlisted, not Private, or markers won't be able to watch it.

Two things from before are still open:

1. **Start command:** confirm it's back to python manage.py init --no-drop && ... in the Render settings. Until it is, each time the service wakes from sleep the data gets wiped.

2. **Push:** commit and push the report changes and the app/cli.py fix.

Then you're ready to export. Send something like *"Build the report. Name: ..., ID: ..."* with your full name and student ID. I'll run the judge, save the Guide chats into docs/transcripts/ with the database password and keys removed, and run python manage.py report to produce the PDF.

Student - 2026-10-12 18:11

https://youtu.be/RPX4wAlBhw8

Guide

That's the same link, and it's already under **YouTube URL** in docs/report.md, so nothing to change.

Two things are still open before you export:

1. **Start command:** is it back to python manage.py init --no-drop && ... in the Render settings? Until it is, each time the service wakes from sleep the data gets wiped.
2. **Push:** commit and push the report and app/cli.py changes.

When both are done, send me your full name and student ID and I'll build the report.

Student - 2026-10-12 18:12

Gian Suarez Nápoles id:816042658

Guide

First I'll check the start command, then load the judge skill and see what the report exporter expects for transcripts.

_tool: mcp__render__get_service_

tool: Bash - Load judge skill and check docs state

tool: Bash - Inspect existing transcripts and session logs

tool: Bash - Survey session logs: time span and student messages

tool: Bash - Check transcript exporter and existing Phase 6 transcript

An earlier Phase 6 chat left a converter (jsonl_to_md.py) that produced 01-08. I'll reuse it so the new files match.

Student - 2026-10-12 18:13

[Request interrupted by user]

Student - 2026-10-12 18:14

just add this last chat

Guide

tool: Bash - Check converter usage and redaction

Only this chat, then. I'll copy the converter, point it at this session only, and add explicit redaction for the secret values that came up here.

tool: Bash - Convert this chat to markdown with secrets redacted

---